Context-Triggered Overrides and Geodesic ETA for Online Multi-Robot Task Allocation under Failures, Mixed Stress, and Deadline Pressure

Abstract

A robot team is rarely handed all of its work up front: work appears, expires, and changes hands while the fleet moves. Online multi-robot task allocation must therefore stay sound as tasks arrive, deadlines tighten, and robots fail mid-execution; no fixed rule serves every regime. We present AHE-MRTA, which selects online among five interpretable allocation behaviours using a two-level rule—deterministic context overrides on failure and deadline pressure, over a dominance dynamic—and prices assignments with a geodesic-ETA oracle and a bounded terminal load repair. We evaluate on three layers of realism: allocation-only, a stochastic navigation proxy (500 seeds), and closed-loop Gazebo/ROS 2 Nav2 with 3, 5, and 10 TurtleBot3 robots (480 runs, 20 seeds per cell at the primary scale). AHE-MRTA leads every scenario on all three layers. Under deadline pressure it leads effective on-time completion (0.650 at 5 robots against 0.578, and 0.764 at 10 against 0.524), cutting the strongest baseline's deadline-violation rate by 17% and 49% while deciding in 0.5–3.4 ms; at the primary scale 20 of 63 cross-method comparisons survive Bonferroni correction, 15 in its favour. A pre-registered matched ablation (240 further runs, four arms, 20 common seeds) then separates what carries that result from what does not: pinning the allocator to a fixed paradigm costs 9–11 percentage points of effective on-time completion (large effect, corrected $p < 0.005$), whereas removing the override cascade changes nothing measurable. We therefore claim online switching over a fixed portfolio, and explicitly not the particular selector we implemented.

Index Terms

Multi-robot systems, task allocation, online algorithms, biomimetic optimization, ROS 2, Gazebo.

I. Introduction

Fielded robot teams are given goals, not schedules. An inspection or delivery mission states what has to be visited and by when; which robot goes where, and in what order, is settled while the mission is already running. That decision is multi-robot task allocation (MRTA). It must respect robot capabilities, task requirements, and operational constraints, and once execution begins it also depends on execution state: a robot may be delayed, unavailable, or committed to a task whose value or feasibility has changed. Recent reviews cover centralized optimization, distributed and market-based methods, metaheuristics, and learning-based policies [1], [2]. Online, then, a low-cost assignment at one instant is not enough: the allocator must preserve useful decisions as the situation moves under it.

No single strategy is best under all of these conditions, because a choice that helps in one regime tends to hurt in another. Three tensions recur. The first is solution quality against reaction cost: centralized formulations jointly optimize assignment and routing when global state is available, folding timing, workload, distance, energy, and deadline terms into one program [3], but re-solving this coupled problem at every event is costly and can overturn a plan that execution has already begun to act on. The second is global optimality against coordination cost: distributed and market-based schemes replace the global solve with local decisions and explicit messaging—group-based auctions serve delivery requests that arrive over time with time windows [4], and consensus-based allocation gives up some optimality for local coordination, speed, and lower energy use [5]—but the locality that bounds their communication also limits how far they exploit global structure. The third is reactivity against stability: an aggressively re-optimizing policy recovers quickly when a robot fails or a deadline slips, yet it preempts running tasks and breaks feasible routes, whereas a commit-once policy is cheap and stable but strands the tasks of a failed robot. Other online methods take fixed positions on these axes—game-theoretic clustering couples task grouping with allocation [6], self-adaptive methods react to shifts in problem structure [7], and decentralized energy-aware policies weight power draw [8]—but each commits to its position in advance. Because arrival bursts, robot failures, and deadline pressure each reward a different position, this motivates treating the allocation behaviour itself as an online choice rather than a fixed design commitment.

This letter considers online single-task-robot allocation under task arrivals, robot failures, mixed stress, and deadline pressure, where the allocator acts at a task arrival, failure, deadline alarm, or replanning event and may use only the state available then. Reassigning a task can reduce estimated travel cost but can also preempt execution, disrupt a feasible route, or move delay to another robot; keeping an incumbent can be equally harmful after a failure or a sharp loss of time slack. We therefore

Manuscript submitted July 2026.

The method keeps the name AHE-MRTA from its implementation, data directories, and the run identifiers of the released campaign. The name is retained for traceability; the contribution claimed here is the context-override cascade, the geodesic-ETA oracle, and the bounded terminal load repair, not the biomimetic metaphor the acronym originally referred to.

take completion and deadline violations as primary outcomes, and report delay, workload balance, recovery, redispatches, preemptions, and decision latency as trade-offs.

Our method, *AHE-MRTA*, is an event-triggered allocator, not a new optimizer at every event. Its behaviour is governed by a two-level selector. The upper level is a deterministic override cascade over observable context: when the recent failure rate exceeds a threshold the allocator switches to orphan-first recovery, and when deadline pressure exceeds a threshold it switches to strict earliest-deadline-first dispatch. The lower level is a dominance dynamic over five interpretable behaviours—spatial greedy, priority-first, deadline-oriented temporal, commit-once stability-oriented, and orphan-first recovery—which decides only when neither override fires. Two further mechanisms shape the cost the selected behaviour minimizes: a geodesic-ETA oracle that prices an assignment by traversable map distance, and a bounded terminal load repair that rebalances the closing schedule under a remaining-makespan non-regression guard.

We state at the outset what our own measurements support, including where they contradicted our expectations. A matched closed-loop ablation (Section VII-E), pre-registered before it was run, shows that the *portfolio* is load-bearing—pinning the allocator to a single fixed paradigm costs 9–11 percentage points of effective on-time completion under deadline pressure, with large corrected effects—while the two-level selector's upper layer is not: deleting the override cascade and falling back to $\arg\max_i D_i$ changes nothing measurable. The two levels are substitutes, not a mechanism and its ornament. We therefore claim online switching over a fixed portfolio and not the specific rule that performs it, and we mark clearly where the evidence stops (Section VII-G). The allocation-only ablation separates the variants too, but disagrees on which way: it puts one fixed paradigm ahead of the full selector. It charges nothing for congestion or recovery latency, which is what the closed-loop experiment adds.

The paper makes four bounded contributions:

1) It specifies a two-level context-override selector for online MRTA that couples fixed context signals to fixed, interpretable allocation behaviours, and quantifies how much each level actually decides.
2) It records the decision and execution quantities needed to separate allocation quality from navigation and recovery effects.
3) It evaluates the policy across robot-team scales, in a navigation-independent simulator and in a ROS 2/Gazebo stack.
4) It reports allocation-only, stochastic navigation-proxy, and closed-loop Gazebo evidence separately, and shows where they agree and where they do not: all three rank the methods alike, but only the closed loop separates the portfolio (load-bearing) from the selector rule (not).

The three layers differ only in how navigation is modelled—deterministic goal success, stochastic position-dependent goal failure, then the full physics and Nav2 pipeline (Section IV-G)—because a strong score where navigation cannot fail does not by itself demonstrate closed-loop navigation or recovery performance. The closed-loop benchmark compares full methods; the causal effect of the selector is isolated separately, by the matched ablation of Section VII-E.

II. Problem Formulation

Event-driven state. Let $\mathcal{R} = \{r_1, \dots, r_n\}$ be the robot set. A requested task τ is described by

$$\xi_\tau = (p_\tau, t_\tau^{\mathrm{a}}, d_\tau, \pi_\tau, c_\tau),$$

where $p_\tau \in \mathbb{R}^2$ is the goal position, t_τ^{a} is its activation time, $d_\tau \in \mathbb{R}_{\geq 0} \cup \{\infty\}$ is its deadline, $\pi_\tau \in \{1, 2, 3\}$ is its priority, and c_τ is its capability requirement. At event time t_k, $\mathcal{T}_k$ contains every activated task that is not completed or irreversibly cancelled. Robot r has position p_r^k, health state $\phi_r^k \in \{\mathrm{alive}, \mathrm{failed}\}$, navigation status, and an ordered queue q_r^k. The queue contains tasks assigned but not completed, including an in-flight task when one exists.

An allocation event is a task arrival, a robot failure, a deadline alarm, or the replanning timer. Let $\mathcal{R}_k^{\mathrm{av}}$ denote robots that are alive, not navigation-stuck, and have remaining queue capacity. Let $\mathcal{I}_k \subseteq \mathcal{R} \times \mathcal{T}_k$ be the set of healthy in-flight robot–task pairs. These pairs are locked: a new allocation event may change future queue entries but does not cancel an executing Nav2 goal.

Feasible event-level decision. The binary variable $x_{r\tau}^k = 1$ means that task τ belongs to robot r's planned queue after event t_k. The feasible set is

$$\mathcal{X}_k = \Big\{ x \in \{0,1\}^{n \times |\mathcal{T}_k|} : \sum_{r \in \mathcal{R}} x_{r\tau}^k \leq 1, \qquad \forall \tau \in \mathcal{T}_k, \tag{1}$$

$$\sum_{\tau \in \mathcal{T}_k} x_{r\tau}^k \leq Q, \qquad \forall r \in \mathcal{R}, \tag{2}$$

$$x_{r\tau}^k = 1 \Rightarrow \big(c_\tau \subseteq \mathrm{cap}(r)\big), \qquad \forall r, \tau, \tag{3}$$

$$x_{r\tau}^k = x_{r\tau}^{k-1}, \qquad \forall (r, \tau) \in \mathcal{I}_k \Big\}. \tag{4}$$

The first constraint gives a task one owner, the second limits queue length, and the third enforces capability compatibility. The final constraint is the in-flight safety lock used in the execution stack. Failed or navigation-stuck robots cannot receive a new task; their unfinished tasks become eligible for redispatch.

Arrival prediction and admissibility. For a candidate pair (r, τ), let $\hat{a}^k_{r\tau}$ be the predicted arrival time obtained from the current queue endpoint, a queue-length delay surrogate, and travel to p_τ. The predictor uses the cached geodesic ETA of Eq. (20) as its route metric. A pair with no route is infeasible. For a finite deadline, the admission rule is

$$\hat{a}^k_{r\tau} > d_\tau + s^k_{r\tau} \quad \Longrightarrow \quad (r, \tau) \notin \mathcal{F}_k, \tag{5}$$

where $\mathcal{F}_k$ is the eligible-pair set. The pair-specific allowance $s^k_{r\tau}$ is zero for new tasks in the nominal configuration. A bounded positive allowance is available only to old rescue tasks and can be widened in failure mode. Thus, the allocator rejects a pair that is incompatible, unavailable, unreachable, or later than the allowance for that task state.

Event-level surrogate. Minimizing non-negative cost over $\mathcal{X}_k$ alone would admit the trivial all-zero assignment because the ownership constraints are upper bounds. The implementation therefore uses a cardinality-first rule. After masking infeasible pairs and preserving locked incumbents, define

$$\mathcal{X}^\star_k = \arg\max_{x \in \mathcal{X}_k} \sum_{r \in \mathcal{R}} \sum_{\tau \in \mathcal{T}_k} x^k_{r\tau}. \tag{6}$$

Among these maximum-cardinality event plans, the selected paradigm minimizes its shaped cost:

$$x^k \in \arg\min_{x \in \mathcal{X}^\star_k} \sum_{r \in \mathcal{R}} \sum_{\tau \in \mathcal{T}_k} C^k_{r\tau} x^k_{r\tau}, \qquad C^k_{r\tau} = \infty \text{ if } (r, \tau) \notin \mathcal{F}_k. \tag{7}$$

Repeated LSA passes implement this lexicographic rule; tasks for which no feasible capacity remains stay open for a later event. The active paradigm then sets queue order (Section III-E). This is an online event-level surrogate, not an offline optimum for the complete mission.

Cost and assignment stability. The implementation evaluates seven normalized features: arrival time $A^k_{r\tau}$, priority penalty $P_\tau = (4 - \pi_\tau)/3$, battery risk B^k_r, relative load L^k_r, failure risk F^k_r, deadline urgency U^k_τ, and navigation/recovery risk R^k_r. For a feasible pair, the common cost is

$$\begin{aligned} C^k_{r\tau} = \rho(\pi_\tau)\big[&w_d A^k_{r\tau} + w_p P_\tau + w_b B^k_r + w_l L^k_r \\ &+ w_f F^k_r + w_t U^k_\tau + w_r R^k_r + \Psi^k_{r\tau}\big], \\ \rho(\pi) = 1 &- 0.1(\pi - 1). \end{aligned} \tag{8}$$

Here $A^k_{r\tau} = \min(1, \hat{a}^k_{r\tau}/A_{\max})$ with $A_{\max} = 220$ s. Because $\hat{a}^k_{r\tau}$ is an absolute time, this feature saturates for every candidate pair once the run passes $A_{\max}$; late in a long run the travel-time channel therefore stops discriminating and the decision is carried by the remaining features and by $\Psi^k_{r\tau}$. In the reported runs (makespans of 150–390 s) this affects the tail of the 10-robot cells. The other base features are dimensionless. The correction $\Psi^k_{r\tau}$ collects the deadline and critical-battery penalties, deadline-capability and bid bonuses, incumbent sticky bonus, reassignment penalty, and capacity penalty. This form avoids adding seconds directly to normalized features. The selector supplies w; the active paradigm can also shape task order and eligibility. A safe incumbent retains base urgency, whereas an unassigned or at-risk task receives nonlinear urgency growth. The factor ρ was written as a discount for urgent work and behaves as one while the bracket is positive. In the evaluated configuration it is not: the deadline-capability bonus inside $\Psi^k_{r\tau}$ outweighs the weighted features, so the bracket is negative for feasible pairs and ρ then charges a priority-3 task *more* than a priority-1 task in the same state. We report the implemented form rather than the intended one and quantify the effect in Section VII-G.

Run-level outcomes and timing contract. The run-level objective is to maximize completion and minimize deadline violations without hiding unfinished tasks. Let $\mathcal{T}^{\text{done}} \subseteq \mathcal{T}^{\text{req}}$ be the tasks completed by $T_{\max}$. Formally,

$$\max \ \text{CR} = \frac{|\mathcal{T}^{\text{done}}|}{|\mathcal{T}^{\text{req}}|}, \qquad \min \ \text{DVR} = \frac{|\{\tau : d_\tau < \infty \ \wedge (t^{\text{c}}_\tau > d_\tau \ \vee \ \tau \notin \mathcal{T}^{\text{done}})\}|}{\max\big(1, |\{\tau : d_\tau < \infty\}|\big)}. \tag{9}$$

The remaining reported quantities—delay, makespan, recovery time, workload balance, redispatches, preemptions, decision latency, messages, and distance—describe the cost of these outcomes. Delay and DVR are evaluated on all requested tasks; an unfinished task is right-censored at $T_{\max}$ for delay and is a deadline miss for DVR (Section IV-H).

For the deployed AHE cycle, the decision-time contract is $\ell_k = t^{\text{publish}}_k - t_k \le 50$ ms. Baseline decision latencies are reported as measured comparison outcomes rather than being silently clipped to this AHE service target. This distinction preserves the real-time requirement while making latency trade-offs visible.

III. AHE-MRTA: A Context-Override Allocator

Design principle and event-triggered architecture.

Section I argued that no fixed allocation policy is best across changing regimes. AHE-MRTA answers this not with a new global solver but with an event-triggered *selection hyper-heuristic* that chooses, online, among a fixed library of five well-understood low-level behaviours [9]. At each allocation event it observes the fleet state, the open task pool, and the previous

queues, updates its context, activates one paradigm, and publishes robot-specific queues. The selector runs no new inter-robot negotiation protocol: the central manager publishes only each robot's own queue, keeping a low-bandwidth interface.

Selection is a two-level cascade, and the levels are not equal partners. The upper level is a pair of deterministic context overrides—one for failure, one for deadline pressure—that fire on threshold crossings and settle three quarters of all events (Section III-B). Only when neither fires does a dominance dynamic over the five behaviours decide (Section III-C); Section VII-G reports how little that tier changes. Two further mechanisms shape the cost the selected behaviour minimizes rather than the choice itself: a cached geodesic-ETA oracle and a bounded terminal load repair (Section III-D). We describe the cascade first because that is where the decisions are made.

The persistent selector state comprises a five-dimensional dominance vector $D_k \in \mathbb{R}^5_{\geq 0}$, the last accepted paradigm, and the remaining dwell counter for that paradigm. The allocator additionally retains in-flight goals, previous task–robot ownership, and an orphan-task pool. This second group is not selector memory: it is needed to enforce the in-flight lock and reassignment safety rules of Section II. Figure 1 summarizes the information flow from an event to queue publication.

A. Context representation

At each allocation event t_k, the observable system state is compressed into the four-dimensional context vector

$$c_k = \left[c_{1,k}, c_{2,k}, c_{3,k}, c_{4,k}\right]^\top \in [0,1]^4. \tag{10}$$

Let $n = |\mathcal{R}|$ be the fleet size and $m_k = |\mathcal{T}(t_k)|$ the number of open tasks. Let $\mathcal{R}_k^{\mathrm{av}}$ denote alive, non-stuck robots with queue capacity, and $\mathcal{R}_k^{\mathrm{f}}$ failed or stuck robots. The components are

$$c_{1,k} = \min\left(1, \frac{m_k}{\max(1,n)}\right), \qquad \text{task density,} \tag{11}$$

$$c_{2,k} = \frac{|\mathcal{R}_k^{\mathrm{av}}|}{\max(1,n)}, \qquad \text{robot availability,} \tag{12}$$

$$c_{3,k} = \frac{|\{\tau \in \mathcal{T}(t_k) : d_\tau - t_k \leq \Delta_d\}|}{\max(1,m_k)}, \qquad \text{deadline pressure,} \tag{13}$$

$$c_{4,k} = \frac{|\mathcal{R}_k^{\mathrm{f}}|}{\max(1,n)}, \qquad \text{failure rate} \tag{14}$$

Here, $\Delta_d = 60$ s. Tasks with an infinite deadline do not enter the $c_{3,k}$ count. The first two signals describe demand–supply balance, the third the fraction of tasks with shrinking time slack, and the last the capacity loss requiring recovery. They are computed from the task pool and already published robot-status summaries; the selector therefore adds no inter-robot communication burden.

B. Context-override cascade

The acute regimes are handled first and deterministically, without consulting any adaptive state. The raw choice $\tilde{p}_k$ follows the priority order

$$\tilde{p}_k = \begin{cases} \text{H\_RECOV}, & c_{4,k} > 0.05, \\ \text{H\_TEMP}, & c_{4,k} \leq 0.05 \ \wedge \ c_{3,k} > 0.50, \\ \arg\max_i D_{i,k+1}, & \text{otherwise.} \end{cases} \tag{15}$$

Recovery therefore has priority when an event contains both failure and deadline pressure. The third branch is a fallback, described in Section III-C; H_TEMP is used instead whenever D_k is absent or its component range is below 10^{-4}, i.e., when that state is not discriminative.

This ordering is not a presentational choice: it is where the decisions are made. Replaying the selector over the 9,989 logged states of the benchmark campaign, the failure override decides 50.2% of events and the deadline override a further 25.0%, leaving 24.8% to the fallback (Section VII-G reports what the fallback does with them). The two thresholds are the method's operative content.

Because the raw choice can switch rapidly, a change other than H_RECOV is held for four allocation events. More specifically, if $\tilde{p}_k \neq p_{k-1}$ while the dwell counter is positive, the dispatcher returns $p_k = p_{k-1}$ and decrements the counter. Once a change is accepted, the counter is reset to $\rho = 4$. H_RECOV bypasses this hold, so a detected failure does not defer the recovery pass to the next event. This mechanism retains override reactivity while limiting ping-pong switching in normal regimes.

TABLE I
CONTEXT PROTOTYPE VECTORS $V_i \in [0,1]^4$ FOR EACH PARADIGM. COLUMN HEADERS: c_1 TASK-DENSITY (TD), c_2 ROBOT-AVAILABILITY (RA), c_3 DEADLINE-PRESSURE (DP), c_4 FAILURE-RATE (FR). HIGH VALUE = PARADIGM EFFECTIVE WHEN THIS CONTEXT DIMENSION IS HIGH.

Hormone	td	ra	dp	fr
H_SPATIAL	0.7	0.7	0.1	0.1
H_CRIT	0.3	0.5	0.8	0.2
H_TEMP	0.5	0.5	0.9	0.1
H_STAB	0.3	0.3	0.3	0.8
H_RECOV	0.3	0.2	0.2	0.9

C. Cost weights and the dominance fallback

Each paradigm $i \in \{1,\dots,5\}$ has a designer-defined context prototype $V_i \in [0,1]^4$. The prototypes are not learned class centres; they are fixed, interpretable priors on which signal combination supports the associated behaviour. Compatibility with the current context is

$$v_{i,k} = \frac{V_i^\top c_k}{\|V_i\|_2 \, \|c_k\|_2} \quad \text{and} \quad \mathbf{v}_k = [v_{1,k},\dots,v_{5,k}]^\top, \tag{16}$$

with zero compatibility if either vector has zero norm. Table I gives the prototypes.

a) Dominance fallback.: When neither override fires, the third branch of Eq. (15) reads a five-dimensional dominance vector D_k, initialized uniformly and kept on the probability simplex at every event ($D_0 = \frac{1}{5}\mathbf{1}$, $D_k \succeq 0$, $\|D_k\|_1 = 1$). Its update

$$\begin{aligned} D_{k+1} = \mathrm{Proj}_{\Delta^4}\Big(\mathrm{clip}_{[0,1]} \big[\alpha D_k + \eta A D_k - \lambda S D_k \\ + \beta \mathbf{p}_k + \gamma \mathbf{v}_k + \delta \mathbf{b}_k \big] \Big) \end{aligned} \tag{17}$$

combines interaction-inspired winner-reinforcement (A) and competitor-suppression (S) terms with context compatibility $\mathbf{v}_k$ (Eq. (16)), short-horizon performance $\mathbf{p}_k = g_k \mathbf{v}_k$ scaled by the completed-minus-failed ratio $g_k = (N_k^{\mathrm{c}} - N_k^{\mathrm{f}})/\max(1, N_k^{\mathrm{c}} + N_k^{\mathrm{f}})$, and a failure support $\mathbf{b}_k = c_{4,k}[-0.3, 0, 0, 0.4, 0.6]^\top$ that lifts H_RECOV and H_STAB while damping the purely spatial behaviour [10]. Here Proj_{Δ^4} is the implementation's non-negative ℓ_1 normalization with a uniform fallback, not an iterative Euclidean projection. The interaction matrices are nearly empty: A has two nonzero entries (H_CRIT supports H_TEMP and H_STAB supports H_RECOV, both 0.20) and S has one (H_TEMP suppresses H_SPATIAL at 0.30); every other entry is zero. Functionally this is a fixed, low-dimensional smoothing of paradigm preference that supplies learning-free hysteresis. It is not an online learned model, carries no optimality guarantee, and—as Section VII-G quantifies—changes the selected paradigm in three of the $9{,}989$ replayed states. We report it because the system runs it, not because the evidence supports it as a contributor. The same vector does carry weight generation below, which is load-bearing.

b) Weight generation.: The selector also produces the seven weights of the common cost function. The channels correspond, in order, to travel time, priority, battery risk, queue load, failure risk, deadline urgency, and reassignment/recovery cost. The fixed paradigm–weight map is

$$M = \begin{bmatrix} 0.9 & 0.1 & 0.1 & 0.3 & 0.3 \\ 0.1 & 0.9 & 0.5 & 0.5 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.3 & 0.3 \\ 0.1 & 0.1 & 0.1 & 0.1 & 0.3 \\ 0.1 & 0.1 & 0.1 & 0.9 & 0.9 \\ 0.1 & 0.5 & 0.9 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.3 & 0.9 \end{bmatrix}. \tag{18}$$

Accordingly,

$$w_k^{\mathrm{eco}} = \mathrm{softmax}\left(\frac{M D_{k+1}}{T_w} \right), \qquad w_k = 0.3 w_0 + 0.7 w_k^{\mathrm{eco}}, \tag{19}$$

where $T_w = 0.3$. When recovery mode is active, w_k is additionally blended toward w_{rec} with $\theta_k = \min(0.80, 0.50 + 0.60 c_{4,k})$. If deadline pressure exceeds 0.5 and recovery mode is inactive, the urgency channel is multiplied by 1.5. These weights are the context-dependent part of the common cost in Section II. The evaluated execution stack has no battery model, so the battery channel carries zero weight in every reported run; it is kept in the cost vector so that the same weight map transfers unchanged to battery-aware deployments.

c) Feasibility and stability.: Every paradigm applies the hard admission gate of Eq. (5) and the single-owner and in-flight constraints. For unassigned or at-risk tasks that have consumed more than 0.7 of their deadline budget, the urgency penalty is increased nonlinearly. A task expected to finish on time with its incumbent owner is exempt. Changing a previous owner requires at least a 10% improvement in estimated arrival; orphan and rescue tasks are exempt from this margin rule. These safeguards prevent event-triggered replanning from becoming gratuitous task migration.

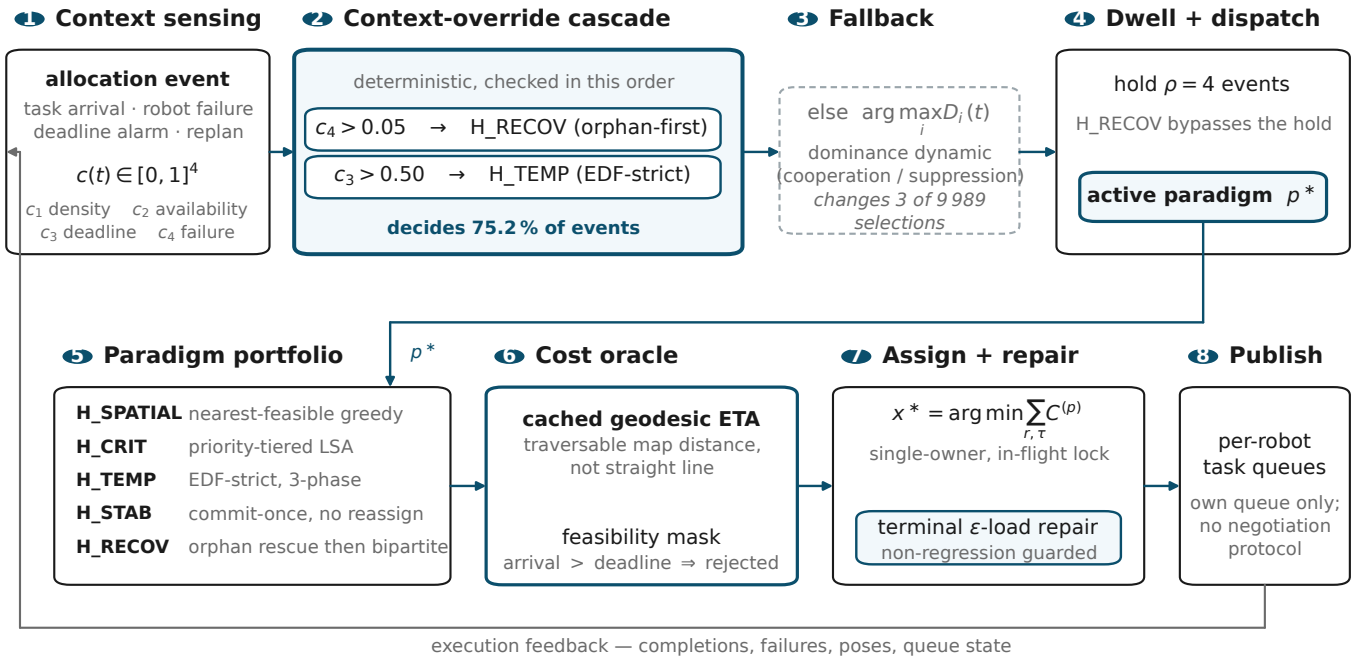


Fig. 1. Two-level context-override selection. Context c_k updates the dominance state; failure/deadline overrides and dwell hysteresis select the active paradigm. The selected paradigm and cost weights produce robot-specific queues through feasibility-masked assignment over the cached geodesic-ETA cost oracle, followed by the terminal bounded load repair. The fixed coefficients are listed in Section III-G.

D. Geodesic ETA and bounded load repair

AHE-MRTA uses a cached geodesic cost oracle and a terminal, bounded load-repair step. Neither component changes the selector or the paradigm library. The cost oracle uses cached geodesic distance d_G on the 0.55 m-inflated occupancy mask of the shared Gazebo map, using an eight-connected free-cell graph. A pair is infeasible if no route exists. For robot r with ordered queue $Q_r = (q_1, \ldots, q_j)$, let $e_r(Q_r) = p_r$ when $j = 0$ and $e_r(Q_r) = p_{q_j}$ otherwise. The implemented ETA surrogate is

$$\hat{a}^k_{r\tau} = t_k + \frac{d_G(e_r(Q_r), p_\tau)}{v_r} + j\,(h + s_\tau), \tag{20}$$

where s_τ is the candidate service time and $h = 22$ s is the fixed per-queued-task navigation allowance. This deliberately inexpensive surrogate does not claim to reconstruct every intermediate route edge. The same endpoint distance oracle is used by the admission gate, paradigm costs, and repair burden calculations.

After the primary plan x^0 is produced, the bounded load-repair step considers local candidates that change the robot only for newly assigned tasks. It therefore never cancels an executing Nav2 goal. Let $M(x)$ be the predicted number of deadline misses, $D_G(x)$ total geodesic distance, $J(x)$ the projected active-robot Jain index, $B_r(x)$ accumulated and committed productive load, and $F(x)$ the maximum predicted remaining completion time. The admissible candidate set is

$$\begin{aligned}\mathcal{N}(x^0) = \big\{x : M(x) \le M(x^0), \quad & D_G(x) \le (1+\epsilon) D_G(x^0),\\ J(x) \ge J(x^0), \quad & \max_r B_r(x) \le \max_r B_r(x^0),\\ F(x) \le F(x^0)\big\}, \quad & \epsilon = 0.10.\end{aligned} \tag{21}$$

Within this set, candidates are ranked by first increasing $J(x)$ and then, in order, decreasing maximum load, load range, and total distance. Repair is enabled only in the terminal phase, when the remaining active-task count is at most three times the healthy-robot count. For an executing first task, $F(x)$ can use Nav2's published remaining-path length instead of a static grid query. These guards prevent the repair from accepting a visually more balanced reassignment that increases deadline misses, route cost, or remaining makespan.

The geodesic ETA and bounded repair change execution cost but leave the selector unchanged; the closed-loop benchmark therefore does not independently test the causal effect of the selector. This distinction is maintained in the interpretation of the results.

TABLE II
FIVE HORMONES, FIVE ALLOCATION PARADIGMS.

Hormone	Paradigm	Inner mechanism
H_SPATIAL	spatial_greedy	nearest-feasible + reject
H_CRIT	priority_first	priority-tiered LSA
H_TEMP	edf_strict	EDF bipartite (multi-phase)
H_STAB	commit_once	hard sticky, no reassign
H_RECOV	orphan_first	orphan rescue + bipartite

Algorithm 1 AHE-MRTA Event Cycle

Require: $\mathcal{R}$, $\mathcal{T}(t_k)$, previous queues, D_k, p_{k-1}, dwell counter h_k; fixed A, S, V, M
$c_k \leftarrow \textsc{ContextVector}(\mathcal{R}, \mathcal{T}(t_k))$
Update the feasibility mask, orphan pool, and in-flight locks
$\mathbf{v}_k \leftarrow (\cos(V_i, c_k))_{i=1}^{5}$; $\mathbf{p}_k, \mathbf{b}_k \leftarrow \textsc{PerformanceAndFailureSupport}()$
$D_{k+1} \leftarrow \textsc{UpdateDominance}(D_k, \mathbf{v}_k, \mathbf{p}_k, \mathbf{b}_k, A, S)$ {Eq. (17)}
$w_k \leftarrow \textsc{GenerateWeights}(D_{k+1}, c_k, M)$ {Eq. (19)}
$\tilde{p}_k \leftarrow \textsc{RawChoice}(c_k, D_{k+1})$ {Eq. (15)}
$(p_k, h_{k+1}) \leftarrow \textsc{ApplyDwell}(\tilde{p}_k, p_{k-1}, h_k)$
$x_k \leftarrow \textsc{Paradigm}_{p_k}(\mathcal{R}, \mathcal{T}(t_k), w_k)$ {Table II}
if in the terminal repair phase **then**
  $x_k \leftarrow \textsc{EpsilonLoadRepair}(x_k, d_G, \epsilon)$
end if
$x_k \leftarrow \textsc{SingleOwnerAndInFlightLock}(x_k)$
Publish robot-specific queues and store (D_{k+1}, p_k, h_{k+1})

E. Paradigm library

The five behaviours share the same feasible set and cost infrastructure; they differ in task-processing order, cost shaping, and treatment of incumbent ownership. Given a feasibility-masked cost matrix $C_k^{(p)}$, the common assignment kernel is

$$x_k^{(p)} \in \arg\min_{x \in \mathcal{X}_k^\star} \sum_{r \in \mathcal{R}} \sum_{\tau \in \mathcal{T}(t_k)} C_{r\tau,k}^{(p)}\, x_{r\tau}. \tag{22}$$

For an infeasible robot–task pair, $C_{r\tau,k}^{(p)} = \infty$. In the implementation, linear sum assignment (LSA) is repeated across task tiers or processing phases. Equation (22) is therefore the surrogate for one assignment pass, not a claim of global optimality for the entire run.

- **H_SPATIAL / *spatial_greedy*:** Tasks are processed in deadline order and assigned to the eligible robot with the smallest expected arrival time. This follows the nearest-feasible rule used in metric MRTA [11], [12].
- **H_CRIT / *priority_first*:** Tasks are partitioned into priority tiers $\pi_\tau = 3, 2, 1$. LSA is applied first to the most critical tier and then to lower tiers, so critical tasks claim capacity first. The design follows the tiered allocation idea in priority-aware auction and consensus strategies [13], [14].
- **H_TEMP / *edf_strict*:** The default temporal behaviour processes tasks in increasing d_τ order and applies the hard deadline gate. The implementation uses a fast pass for near-deadline tasks, a recovery pass for any current orphans, and an LSA pass for the remaining tasks. Queue order may be improved by cheapest-insertion candidates that preserve deadline feasibility. The behaviour applies EDF to deadline-constrained MRTA [15], [16].
- **H_STAB / *commit_once*:** Tasks with a previous owner that remains eligible are kept with that owner. Only new tasks receive a single-pass allocation. This lowers reassignment churn but limits flexibility after a failure, making the stability trade-off explicit.
- **H_RECOV / *orphan_first*:** Tasks left by failed, stuck, or navigation-aborted robots are first redistributed among healthy robots through LSA; remaining tasks then receive temporal allocation. Recovery therefore obtains a dedicated decision pass before new work [12], [17].

F. Algorithm

Algorithm 1 gives the execution order at one allocation event. A task arrival, robot failure, deadline alarm, or replanning timer triggers the cycle. Only queues are published at the end: D_k, A, S, V, and the full cost matrix remain at the manager node.

G. Configuration

The same selector settings are used at all scenarios and fleet scales: $\alpha = 0.65$, $\beta = 0.40$, $\gamma = 0.20$, $\eta = \lambda = 0.12$, $\delta = 0.20$, $T_w = 0.3$, and $\rho = 4$. The base cost profile is $w_0 = (0.34, 0.10, 0.04, 0.16, 0.10, 0.22, 0.04)$ and the recovery profile is $w_{\text{rec}} = (0.55, 0.04, 0.03, 0.22, 0.09, 0.05, 0.02)$, in the channel order given under *Weight generation*. These quantities are neither relearned nor retuned by event or scenario.

Context, dominance, override, and dwell updates have fixed dimension and are therefore $O(1)$. The dominant cost is the LSA call within a paradigm, whose worst-case complexity for one square assignment pass is $O\big(\max(n, m_k)^3\big)$. The geodesic cache reduces repeated goal queries, and load repair is restricted to a fixed number of local move passes. Computational load thus stems from the selected paradigm and optional route/repair layers rather than from the selector itself.

IV. Experimental Design

A. Hardware Platform

All experiments run on an Intel Core i7-11800H workstation with eight physical cores, 16 threads, a 2.30 GHz base clock, 16 GiB DDR4 RAM, Ubuntu 24.04 LTS, and Linux 6.17. The machine has an NVIDIA RTX 3050 Mobile GPU, but no proprietary driver is installed. Gazebo and RViz therefore use Mesa llvmpipe software rasterization (`LIBGL_ALWAYS_SOFTWARE=1`, `GALLIUM_DRIVER=llvmpipe`). No experiment uses GPU acceleration. The reported comparison is CPU-bound.

B. Software Stack

We use **Gazebo Harmonic** (gz-sim 8.x) [18], **ROS 2 Jazzy** [19], and **Nav2** [20]. Each robot is a **TurtleBot3 Waffle Pi** [21] with a 360-ray two-dimensional lidar (5 Hz, 8 m range), a 200 Hz IMU, and a differential-drive controller ($v_{\max}$=0.26 m/s, $\omega_{\max}$=1.82 rad/s).

Each robot has an independent Nav2 lifecycle in its own ROS 2 namespace. The stack uses global and local costmaps with a 0.40 m inflation radius, a Regulated Pure-Pursuit controller, and a behavior-tree navigator. We provide ground-truth localization through a static `map`→`odom` transform anchored at each robot's spawn pose. This is an experimental control: it isolates allocation effects from localization error (Section VII-G). The `ros_gz_bridge` relays `/scan`, `/odom`, `/cmd_vel`, `/joint_states`, and TF for each robot. A TF relay aggregates the namespaced trees for visualization. All methods use identical Nav2 parameters. Figure 2 shows how the ecosystem manager and the per-robot Nav2 lifecycles are wired together in this stack.

a) Evaluated configuration and artifacts.: AHE-MRTA* is evaluated with its geodesic-ETA oracle and terminal load repair active. Both are switched on explicitly at launch: the allocator's built-in defaults reproduce the earlier Euclidean, repair-free reference kept for ablation, so the configuration—not only the method name—determines what runs. The resolved switch set is recorded in each run's metadata and in the campaign directory, so a run cannot be confused with a differently configured one under the same name. The allocator, the campaign drivers, and the scripts that generate every table and figure from the raw logs accompany this submission.

C. Inspection Arena

The arena is a 20×20 m walled indoor mock-up. Two horizontal walls with a 1.5 m central passage create upper and lower lanes. Two rows of square pillars (0.5×0.5 m) line the side walls, and four cylindrical obstacles (r=0.35 m) lie in the lanes. We define 42 fixed inspection waypoints. The waypoints are obstacle-aware and at least 1 m apart. Each run samples its task set deterministically from this grid using its seed. Figure IV-C shows the arena, spawn positions, and sampled task sets.

D. Evaluation Scales

We evaluate three Gazebo scales (Table III). Each scale contains 60 runs: 4 methods × 3 scenarios × 5 seeds.

TABLE III
Gazebo Evaluation Scales

Scale	Robots	Tasks	Density	Total exps
3r-low	3	9	3.0 t/r	60
3r-mid	3	15	5.0 t/r	60
3r-high	3	24	8.0 t/r	60
5r-mid	5	25	5.0 t/r	60
10r-mid	10	50	5.0 t/r	60
Total Gazebo experiments				**300**

At N>3, some per-robot `odom`→`base_link` transforms arrived after costmap activation. We resolve this lifecycle race by delaying activation for robot i by $6i$ s at 3r and 5r and by $20i$ s at 10r. We also broadcast an identity transform at node startup. All scales use the same arena, obstacle map, and Nav2 parameters.

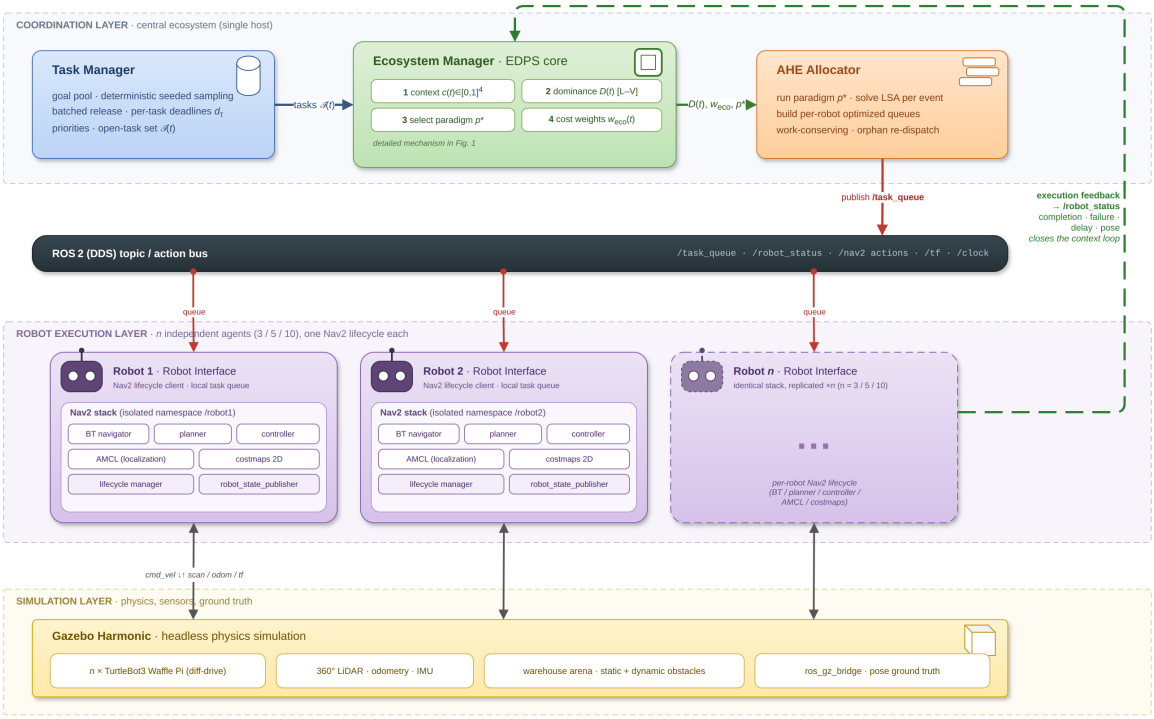


Fig. 2. System architecture. The ecosystem manager updates the hormone state, selects the active paradigm, and publishes one task queue per robot through ROS 2 topics. Each robot runs its own Nav2 lifecycle and returns completion, failure, and delay feedback to the manager.

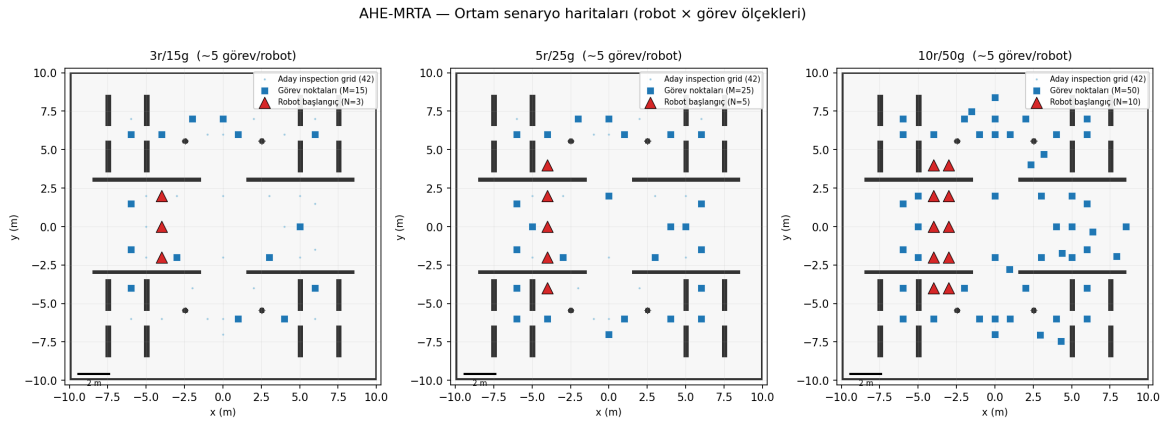


Fig. 3. Arena configurations by scale. **Left:** 3r/15t, with robots in one central column. **Centre:** 5r/25t, with two spawn columns. **Right:** 10r/50t, with all five columns active. Blue stars are task goals for seed 1; colored triangles are robot spawns. All waypoints are obstacle-free.

E. Stress Scenarios

The three scenarios stress different allocation requirements:

Every run has the same horizon $T_{\max} = 900$ s; a task still open at $T_{\max}$ is unfinished (Section IV-H). In all three scenarios each task draws an independent priority $\pi_\tau \sim \mathcal{U}\{1, 2, 3\}$, a service time $\sim \mathcal{U}[2, 8]$ s, and a deadline $d_\tau = t_0 + \Delta_\tau$ measured from the run start t_0; the scenarios differ in the deadline budget Δ_τ, the task-arrival schedule, and whether a robot fails.

- **robot_failure:** all tasks are released at t_0 with a nominal budget $\Delta_\tau \sim \mathcal{U}[90, 300]$ s. One designated robot is disabled at $t = 45 \pm 5$ s (uniform jitter, drawn per seed), early enough that it is carrying work. Its assigned tasks become orphaned and must be redistributed. This scenario tests reactive recovery and the orphan-rescue behaviour.
- **mixed_stress:** the same failure event is combined with staggered demand and a tighter budget: tasks arrive in three waves at $t = 0$, 30, and 60 s ($\lfloor M/3 \rfloor$, $\lfloor M/3 \rfloor$, and the remainder), and the budget is halved ($\Delta_\tau \sim \mathcal{U}[45, 150]$ s). Because deadlines run from t_0, later waves arrive with less slack, so the allocator must absorb arrivals, priority spread, and a failure at the same time.
- **deadline_pressure:** no robot fails and all tasks are released at t_0, but the deadline budget is cut to 40% ($\Delta_\tau \sim \mathcal{U}[36, 120]$ s).

The lower end of this range is below the Nav2 travel time to the farthest waypoint at $v_{\max}$, so part of the set is unservable on time by construction and the scenario separates urgency-driven dispatch and EDF ordering.

Failure injection targets a fixed robot index rather than a random one, so that the recovery burden is identical across methods for a given seed; task positions, priorities, service times, deadline draws, and the injection time are all seed-controlled and shared by all four methods.

F. Baseline Methods

We compare one operational adaptation from each online-MRTA family represented in the AHE-MRTA library. All adaptations run in the same framework. They share the event loop, ROS 2 topic interface, cost inputs (arrival time, urgency, priority, and load), and Nav2 client. Only the allocation policy differs. The per-robot queue cap Q of Eq. (2) is part of that policy rather than a shared constant: BiG-MRTA and Consensus-DBTA hold $Q{=}5$, RoSTAM-EA is unbounded within the event, and AHE-MRTA deliberately runs shallow at $Q{=}2$ (raised by one in the terminal repair phase) so that queued work stays reassignable after a failure. We use source-paper parameters where they are specified and otherwise report fixed benchmark choices. The names below thus identify our implementations, not independent reproductions of the complete source systems.

a) BiG-MRTA [22] (bipartite matching).: Our BiG-MRTA adaptation builds a bipartite robot–task graph with fixed multi-criteria edge utility and solves maximum-weight matching at each event. Once assigned, a task is not reallocated. It is the deterministic, lowest-latency, single-paradigm comparison. Its policy avoids replanning but does not recover a task when its assigned robot fails or time slack collapses.

b) RoSTAM-EA [7] (evolutionary).: Our RoSTAM-inspired EA evolves candidate assignment vectors with tournament selection, crossover, and mutation at each allocation event. It dispatches the elite individual. It represents self-adaptive stochastic search. It searches a richer assignment space than one-shot matching, but its decision latency is two to three orders of magnitude higher. Its reoptimization does not encode the cause of a context change.

c) Consensus-DBTA [5] (DBTA2-style consensus bidding).: Each robot shares its two highest-valued task bids; simulated network-wide maximum consensus identifies a winner for each task, and priority rounds prevent one robot from winning conflicting tasks. Its bidding rounds keep assignments largely stable, though it still redispatches after a failure. It represents distributed auction/consensus and is the strongest completion-rate baseline in this study. Re-bidding rounds can delay response to a failure or rapid deadline escalation.

The external baselines are not substitutes for a selector ablation: they differ from AHE in allocation logic as well as paradigm choice. The separate fixed-paradigm proxy ablation in Section VII-D addresses only the selector question in the proxy setting.

d) Comparison scope.: The baseline implementations share the event loop, interfaces, and cost inputs, but they are operational re-implementations rather than independent replications of every source system. The four-method benchmark therefore compares these implementations in one common stack. It is not a universal ranking of the cited algorithms. A controlled Gazebo selector ablation against fixed paradigms is required before any closed-loop difference can be attributed to the selector itself (Section VII-G).

G. Evaluation settings

We use three evaluation settings that differ only in how navigation is modelled, so that allocation quality is never conflated with navigation or recovery behaviour. **(i) Navigation-independent** (allocation-only): every dispatched goal is reached deterministically along the shared geodesic map—no local-planner stall, costmap rebuild, or goal failure—and the robot pose advances to each completed goal, while injected robot failures still occur. This setting isolates assignment and queue-ordering quality from navigation noise. **(ii) Stochastic navigation proxy**: navigation cost is a fixed edge, but a goal can fail position-dependently and must be re-attempted, so the metric also rewards resilience to navigation failure. **(iii) Closed-loop Gazebo/ROS 2**: the full physics and Nav2 pipeline above runs end to end. Results from the three settings are tabulated separately and never pooled.

H. Metrics

We report completion rate (CR), deadline violation rate (DVR), makespan, all-task average delay, failure-recovery time, workload balance (Jain), task redispatch rate, execution preemptions, decision latency, and message count. We also report total travel distance where applicable. CR and DVR are the primary outcomes. The other metrics describe efficiency and trade-offs.

a) All-task delay and DVR.: A completed-only average rewards a policy that abandons difficult work. Distant, late, or contended tasks then disappear from the denominator. We avoid this survivorship bias by scoring delay and DVR over *all* requested tasks. Let $\mathcal{T}$ be the requested set, t_τ^{a} the activation time, t_τ^{c} the completion time, and $T_{\max}$ the run horizon. Delay for an unfinished task is right-censored at the horizon:

$$\delta_\tau = \begin{cases} t_\tau^{\mathrm{c}} - t_\tau^{\mathrm{a}}, & \tau \text{ completed}, \\ T_{\max} - t_\tau^{\mathrm{a}}, & \tau \text{ unfinished}, \end{cases} \qquad \bar{\delta} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \delta_\tau, \tag{23}$$

TABLE IV
AHE-MRTA NAVIGATION-INDEPENDENT ALLOCATION-ONLY RESULTS (100 SEEDS PER CELL). NAVIGATION SUCCEEDS DETERMINISTICALLY; DISTANCE IS THE GEODESIC PATH LENGTH ON THE SHARED INFLATED OCCUPANCY MAP.

Scale	Scenario	Fit.	CR	DVR	Jain$_{active}$	Jain$_{dist}$	Delay (s)	Distance	Decision (ms)
3r/15t	Robot failure	1.000	1.000	0.000	0.990	0.830	40.1	154.3	0.056
3r/15t	Mixed stress	1.000	1.000	0.000	0.991	0.833	39.9	153.3	0.042
3r/15t	Deadline pressure	1.000	1.000	0.000	0.986	0.965	65.8	155.5	0.042
5r/25t	Robot failure	1.000	1.000	0.000	0.983	0.841	30.6	183.7	0.072
5r/25t	Mixed stress	1.000	1.000	0.000	0.983	0.843	30.7	183.8	0.059
5r/25t	Deadline pressure	1.000	1.000	0.000	0.978	0.953	60.5	235.7	0.052
10r/50t	Robot failure	1.000	1.000	0.000	0.961	0.859	30.9	272.7	0.361
10r/50t	Mixed stress	1.000	1.000	0.000	0.968	0.864	30.9	277.0	0.356
10r/50t	Deadline pressure	1.000	1.000	0.000	0.977	0.946	56.8	436.0	0.259

Any task with a deadline is a violation if it completes late *or* never completes:

$$\mathrm{DVR} = \frac{\left|\{\tau : d_\tau < \infty \ \wedge \ (t^{\mathrm{c}}_\tau > d_\tau \ \vee \ \tau \text{ unfinished})\}\right|}{\max\left(1, \left|\{\tau : d_\tau < \infty\}\right|\right)}. \tag{24}$$

We apply this convention to every method and scale. We also record completed-only delay and DVR. The two definitions coincide when CR$=1$ and differ only when a method leaves tasks unserved.

I. Statistical Analysis

We use pairwise, two-sided Mann–Whitney U tests with Bonferroni correction within each scenario family (three baselines $\times$ seven metrics, or 21 tests per scenario). At the primary 5-robot scale, each cell has $n=5$ runs (one density and 25 tasks). The 3-robot scale pools three densities ($n=15$), and the 10-robot scale has $n=5$. We also report Cliff's δ, an effect-size estimate suited to small cells. The four-method Gazebo cells with $n=5$ are descriptive. We make no cross-method superiority claim from an uncorrected or non-significant comparison. The 100-seed-per-cell allocation-only campaign and the 500-seed primary navigation-proxy cells are separate controlled tests. Gazebo results describe behavior in the evaluated execution stack, not general deployment performance.

V. ALLOCATION-ONLY AND NAVIGATION-PROXY RESULTS

In the navigation-independent setting (Sec. IV-G), the allocator and execution oracle both use geodesic distance on the same inflated occupancy map, so the metrics measure allocation, queue order, and recovery after failure rather than local-planner noise.

Table IV shows that AHE-MRTA has fitness$=$CR$=1$ and DVR$=0$ in all nine scale–scenario cells (100 seeds per cell). The active-robot Jain index is 0.986–0.991 at 3r, 0.978–0.983 at 5r, and 0.961–0.977 at 10r. Decision latency is at most 0.361 ms, in the largest 10r/50t cell. During development, six 10r seeds allowed one task to execute on two robots, which inflated CR to 1.02. After adding the in-flight owner lock, we reran those seeds and the complete 10r campaign. CR is then exactly 1.000 in every cell. Because completion and deadline metrics are already at the ceiling, this setting does not separate the leading policies; it establishes that AHE-MRTA attains full, deadline-clean coverage with short geodesic routes.

A. Separate resilience experiment: navigation-proxy

The next experiment simplifies navigation *cost* to a fixed-time edge in the robot–task graph. Navigation *outcomes* remain position-dependent and stochastic: a goal can fail and must be reattempted. This is not a navigation-independent setting. We use it as a navigation proxy for recovery after failed goals. Fitness is the priority-weighted fraction of tasks completed before their deadline. Because a failed goal must be recovered, fitness captures both allocation and resilience to navigation failure. With 500 seeds per scenario, AHE-MRTA leads every scenario on both planes (Table V), and eight of the nine paired comparisons against a baseline are significant, most of them overwhelmingly (p from 10^{-9} to 10^{-76}). The exception is a tie with BiG-MRTA under robot failure ($+0.004$, $p=0.14$).

At the primary scale (25 tasks and 5 robots; Table V, left), no method reaches the ceiling: with the deadline budgets the scenarios actually specify, on-time completion separates the four policies even when navigation cannot fail. AHE-MRTA leads at 0.994, 0.727, and 0.840 for robot failure, mixed stress, and deadline pressure, against 0.980, 0.668, and 0.733 for the best baseline in each. When navigation becomes stochastic (Table V, right) every method falls further, and the ordering is

TABLE V

ALLOCATION FITNESS (PRIORITY-WEIGHTED ON-TIME COMPLETION), 500 SEEDS, 5 ROBOTS / 25 TASKS. **LEFT:** PERFECT-NAVIGATION PLANE, WHICH ISOLATES ALLOCATION QUALITY. **RIGHT:** THE STOCHASTIC NAVIGATION PROXY, WHERE NAV2 GOAL FAILURE CAN ALSO COST A TASK. AHE-MRTA LEADS EVERY SCENARIO ON BOTH PLANES; EVERY PAIRED-WILCOXON COMPARISON AGAINST A BASELINE IS SIGNIFICANT EXCEPT ONE (ALL $p < 0.143$ EXCEPT AHE–BIG UNDER ROBOT FAILURE, WHICH IS A TIE). **BOLD** MARKS THE BEST PER COLUMN.

	Perfect nav (alloc. quality)			**Stochastic nav (resil.)**		
Method	RF	MS	DP	RF	MS	DP
AHE-MRTA*	**0.994**	**0.727**	**0.840**	**0.384**	**0.263**	**0.349**
BiG-MRTA	0.980	0.668	0.733	0.380	0.229	0.284
RoSTAM-EA	0.935	0.643	0.679	0.333	0.225	0.227
Cons-DBTA	0.978	0.642	0.729	0.364	0.217	0.258

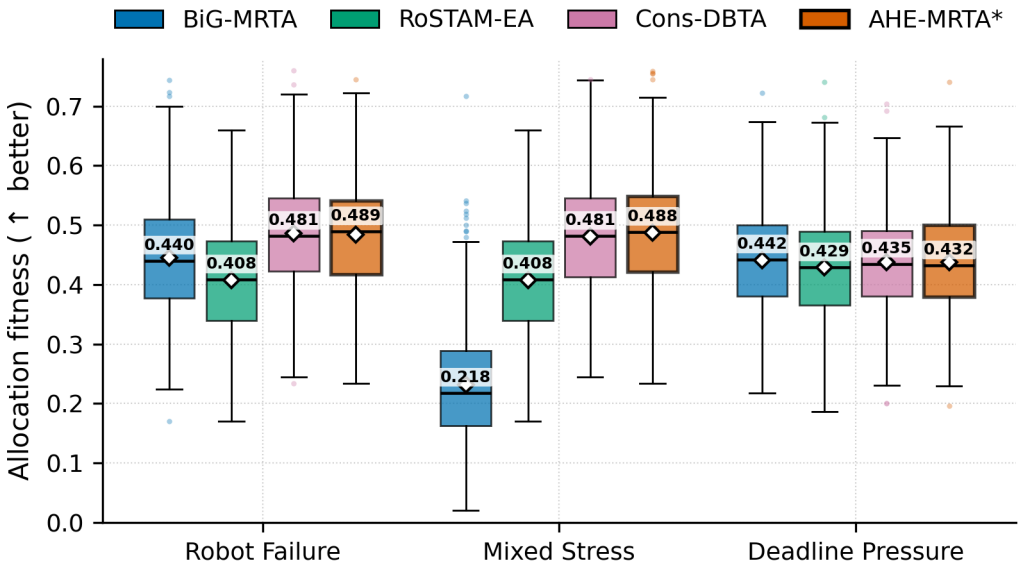


Fig. 4. Navigation-proxy fitness at the primary 5-robot/25-task scale (500 seeds per cell). Boxes show the interquartile range, centre lines the median (value annotated above each box), diamonds the mean, and whiskers 1.5 interquartile ranges; points are outliers. AHE-MRTA leads every scenario; the paired-Wilcoxon comparisons against each baseline are significant in eight of nine cases, the exception being a tie with BiG-MRTA under robot failure (p=0.14).

preserved: 0.384, 0.263, and 0.349 against 0.380, 0.229, and 0.284. The margin is smallest under robot failure, where BiG-MRTA's commit-once discipline is well matched to the regime and the difference is not significant; it is largest under deadline pressure, which is the regime the overrides and the geodesic ETA both target.

An earlier version of this evaluation reported these two planes as saturated and unable to separate the leading policies. That was an artefact of the simulator's scenario parameters having drifted from the ones documented in Section IV-E: deadline budgets were drawn from $\mathcal{U}[200, 400]$ s where the design specifies $\mathcal{U}[36, 120]$ s under deadline pressure, and the halved budget and staggered arrivals of mixed stress were absent. Under budgets that generous every policy completes almost everything on time, so nothing could separate. Both planes now read their scenario definitions from the same module the Gazebo runner uses, and a parity test fails if the two ever diverge again. The proxy still does not isolate which mechanism produces the margin, and it does not predict a closed-loop Nav2 result: it charges no cost for congestion, replanning latency, or recovery time, all of which the physical stack does. Section VI reports the Gazebo estimates separately, and Section VII-E is where the mechanism question is settled. Figure 4 summarizes the proxy results.

B. Navigation-proxy scalability

We examine scalability in two settings. In the navigation proxy, the simplified arrival model permits a high-power sweep over robot count. We run four methods at $N \in \{3, 5, 10\}$ robots, with five tasks per robot ($M = 5N$), 100 seeds per cell, and all three scenarios. This produces 3,600 runs. We also evaluate closed-loop Gazebo execution at 3r, 5r, and 10r. Table VI and Figure 5 average proxy results over scenarios.

TABLE VI
SCALABILITY (STOCHASTIC NAVIGATION PROXY, GEODESIC EXECUTION ORACLE, 100 SEEDS, FIXED DENSITY 5 TASKS/ROBOT, MEAN OVER SCENARIOS). FITNESS ↑, LATENCY (MS) ↓. BEST PER SCALE **BOLD**. PHYSICAL GAZEBO VALIDATION: 3R, 5R, AND 10R CONFIRMED.

	3 robots		5 robots		10 robots	
Method	Fit.	Lat.	Fit.	Lat.	Fit.	Lat.
AHE-MRTA*	**0.300**	0.12	**0.329**	0.20	**0.326**	0.54
BiG-MRTA	0.267	0.02	0.293	0.05	0.298	0.17
RoSTAM-EA	0.249	1.93	0.261	3.48	0.245	7.00
Cons-DBTA	0.257	**0.02**	0.276	**0.03**	0.290	**0.05**

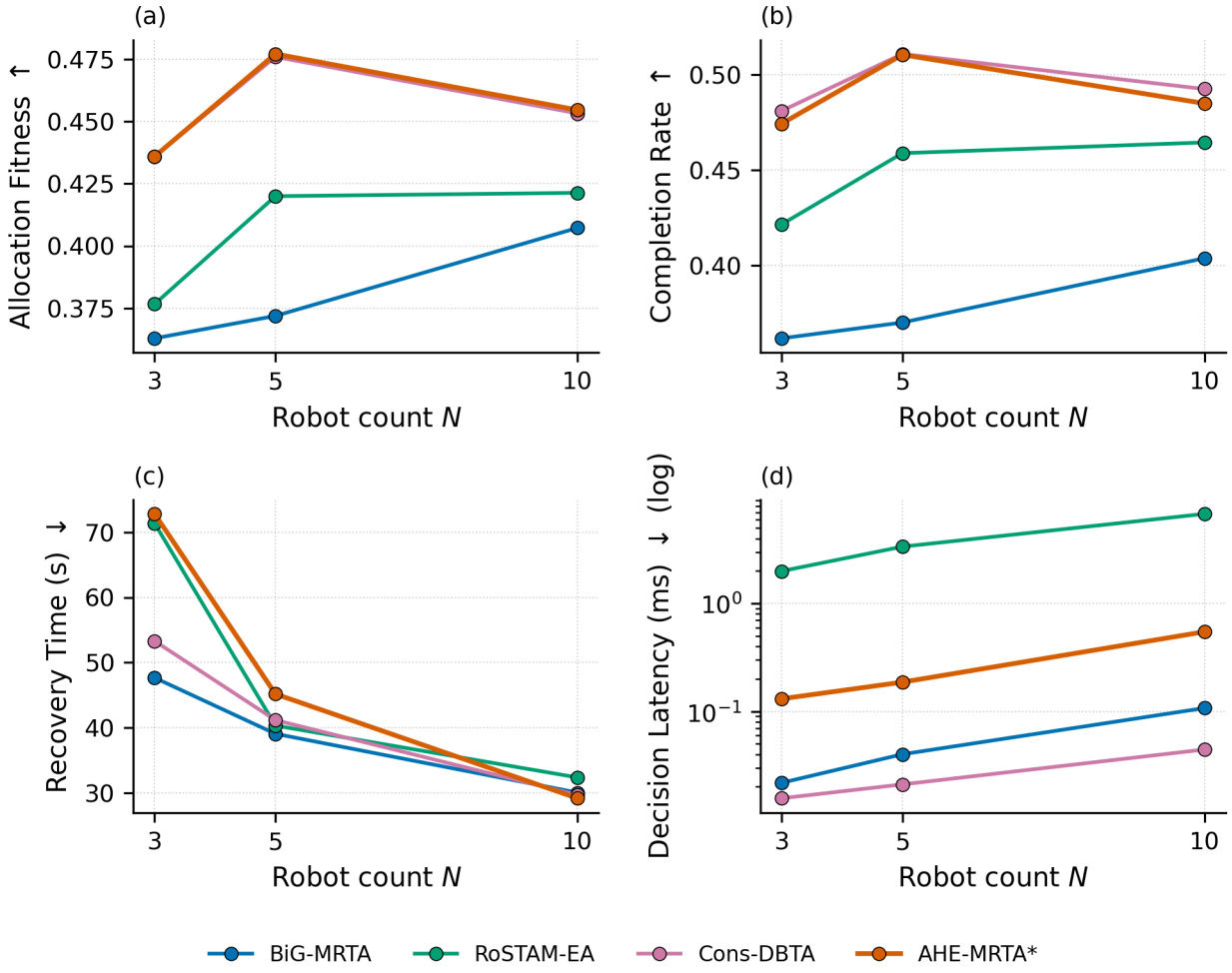
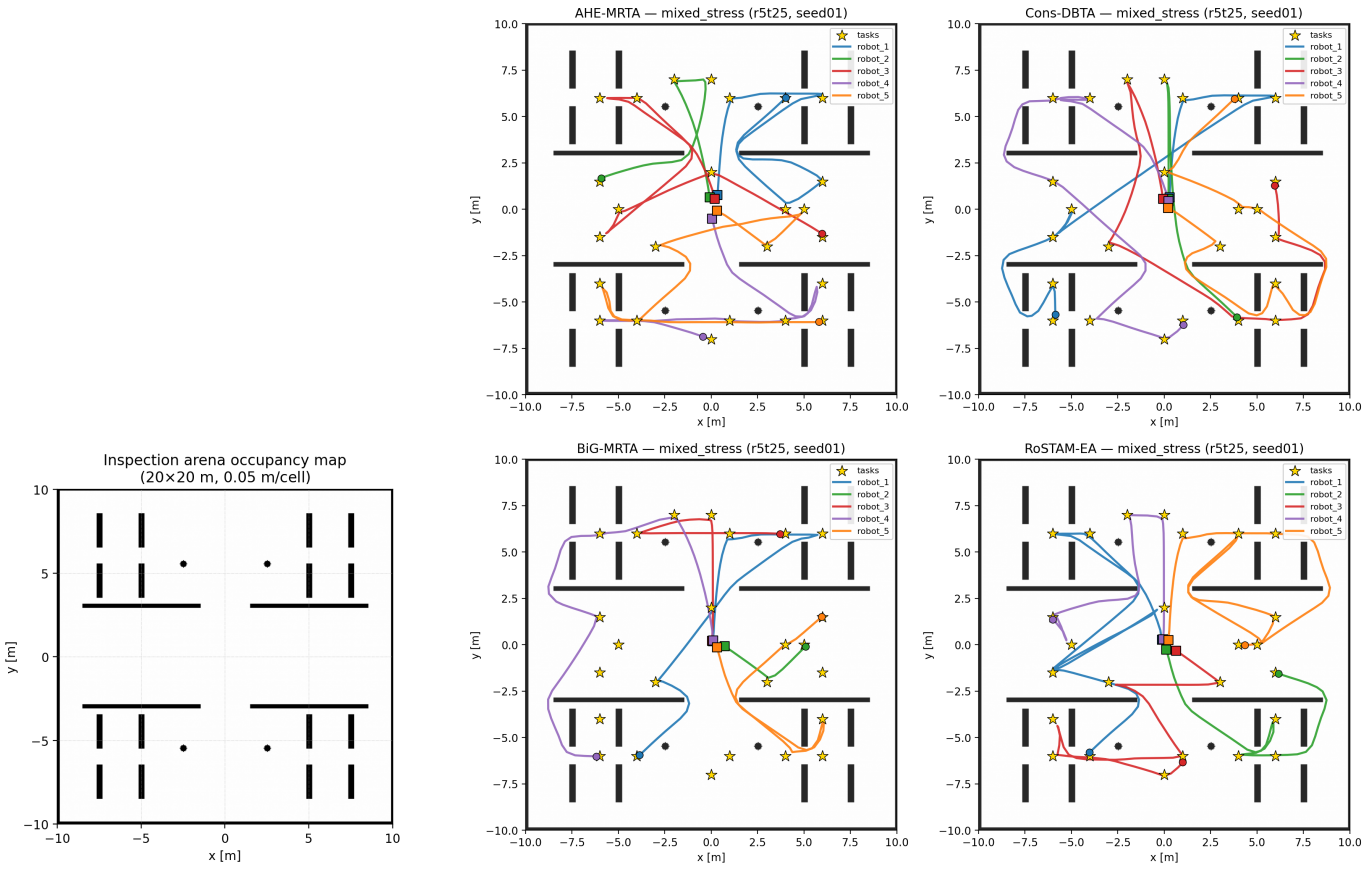


Fig. 5. Scalability by robot count in the stochastic navigation proxy. Values are means over three scenarios with 100 seeds per cell: (a) fitness, (b) completion rate, (c) recovery time, and (d) decision latency. AHE-MRTA is tied with or numerically close to Consensus-DBTA on fitness; the small proxy differences are not treated as separation.

AHE-MRTA is highest on proxy fitness at every scale (Table VI), by 3.3, 3.6, and 2.7 pp over the best baseline at $N = 3$, 5, and 10; the runner-up is BiG-MRTA at all three. The margin does not grow with fleet size, so we read the result as a consistent advantage rather than a scaling one. Proxy decision latency remains about 0.54 ms at 10 robots. This is nearly four orders of magnitude below the 5 s replanning period and about 13 times lower than RoSTAM-EA (7.0 ms at $N = 10$). In the closed-loop Nav2 stack the same allocator decides in 0.5–3.4 ms, once the geodesic oracle is built at start-up rather than inside the first decision (Section VI).

The 5- and 10-robot Gazebo studies contain 60 runs each. Their per-scenario results appear with the 3-robot benchmark in Section VI. They are not a direct confirmation of the proxy because they use a different noise model and fewer seeds.



(a) Pre-built Nav2 occupancy map

(b) Executed ground-truth trajectories, four methods

Fig. 6. Experimental map and representative trajectories. **(a)** The static Nav2 occupancy map (`obstacle_map`) is 20×20 m with 0.05 m cells and origin $[-10, -10]$. It contains divider walls, square pillars, and four cylindrical obstacles. **(b)** Paths from one *mixed_stress* run (5 robots, 25 tasks, seed 1) for AHE-MRTA, Consensus-DBTA, BiG-MRTA, and RoSTAM-EA (clockwise from top left). Squares are spawn positions, gold stars are sampled waypoints, and colored curves are paths reconstructed from ground-truth poses. This single-run figure is qualitative; it does not establish an aggregate method ranking.

VI. Gazebo Benchmark Results

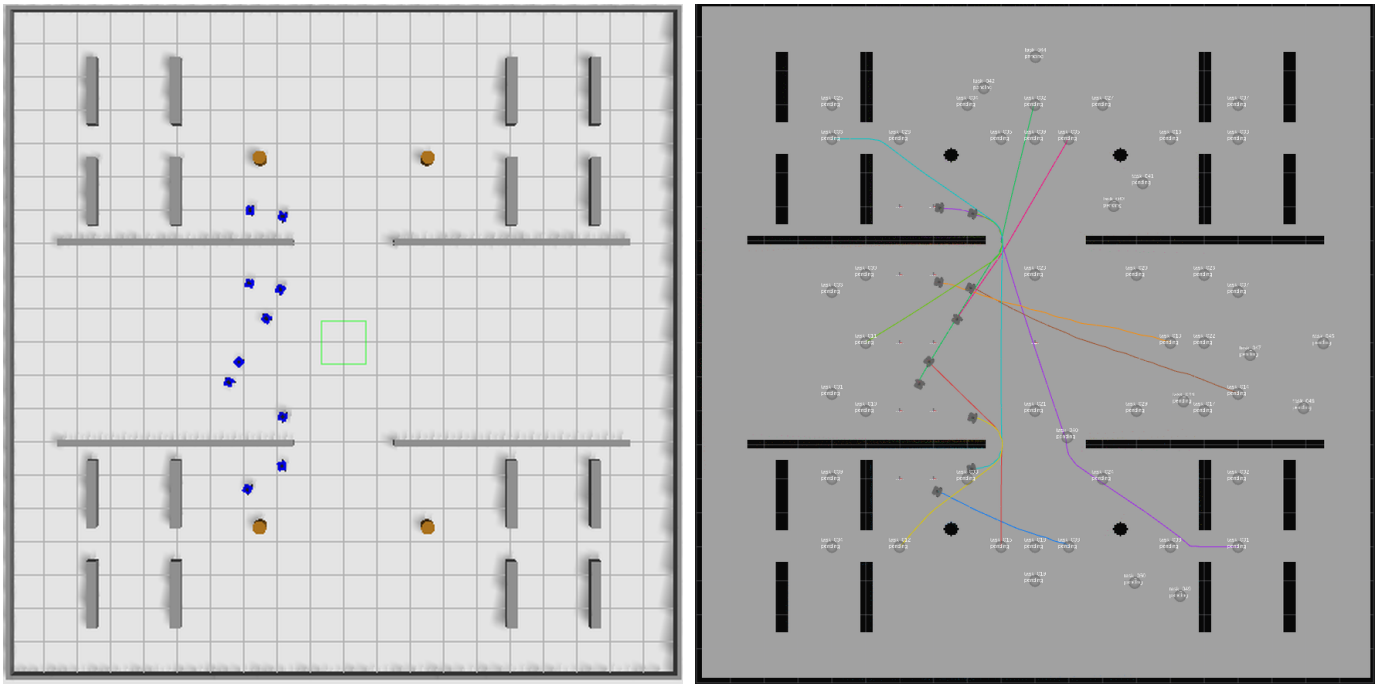
We report the closed-loop Gazebo benchmark. Methods use identical Nav2 parameters, world, and timing budgets; only the allocation policy differs.[1] At 3r, three density levels (9, 15, and 24 tasks) with five seeds per method–scenario combination give 180 runs. The 5r and 10r studies add 60 runs each, for 300 runs in total. We use Bonferroni-corrected Mann–Whitney U tests as described in Section IV-H.

Figure 6 shows the occupancy map and representative trajectories from one *mixed_stress* instance (5 robots, 25 tasks, seed 1). We reconstruct trajectories from logged ground-truth poses and overlay them on the arena and sampled waypoints. In this example, AHE-MRTA and Consensus-DBTA distribute robots across the arena and cover every waypoint. Commit-once BiG-MRTA leaves a larger area uncovered. This is a qualitative illustration, not an aggregate comparison.

Figure 7 shows a mid-run snapshot of the 10-robot configuration under AHE-MRTA. The snapshot comes from an illustrative 10-robot / 50-task run with staggered arrivals and no injected failure; it is outside the three benchmark scenarios and contributes to no reported number. The Gazebo view shows robots travelling to assigned waypoints. The concurrent RViz view shows the occupancy map, each robot's Nav2 global plan, and its ground-truth pose. The snapshot illustrates work-conserving dispatch across both lanes; it is not a quantitative result.

a) The deadline-pressure result.: The clearest closed-loop separation is under *deadline_pressure*, and it is the same at both physical scales. Effective on-time completion, $\mathrm{CR}\times(1-\mathrm{DVR})$, is 0.79 for AHE-MRTA* at 5 robots against 0.66 for the strongest baseline, and 0.76 against 0.52 at 10 robots. The margin comes from deadline compliance rather than from throughput: three of the four methods complete essentially every task at 5 robots, and the separation is entirely in DVR, which AHE-MRTA* holds at 0.350 where the strongest baseline (Consensus-DBTA, the best baseline by the same measure) records 0.422—a 17% relative reduction. At 10 robots the same comparison is 0.220 against 0.428, a 49% reduction, and the baselines reach their completion rates only by tolerating those violations. This is the pattern the method was built for: the deadline override and the geodesic ETA both act on time slack, and this is the regime where slack is scarce.

[1]Tables and figures denote AHE-MRTA as AHE-MRTA* where it is evaluated with its geodesic-ETA cost oracle and terminal load repair (Sec. III-D).



(a) Gazebo Harmonic — 3-D physics

(b) RViz — map + live Nav2 plans

Fig. 7. Representative live snapshot of an illustrative 10-robot / 50-task AHE-MRTA run with staggered task arrivals, outside the three benchmark scenarios. Both panels are captured from the same paused simulation instant of the same run, so robot positions correspond one-to-one. **(a)** Gazebo Harmonic (top-down view) shows the ten TurtleBot3 Waffle Pi robots travelling in the 20 × 20 m arena. **(b)** RViz shows the occupancy map, current Nav2 global plan for each robot, and ground-truth-anchored poses. The image illustrates the execution state of one run; it is not comparative evidence.

TABLE VII

MAIN COMPARISON AT THE PRIMARY 5-ROBOT / 25-TASK SCALE; n=5 RUNS (SEEDS) PER CELL. BEST VALUE PER COLUMN (WITHIN SCENARIO) IN **BOLD**. SIGNIFICANCE VS AHE-MRTA*: $^{*}p<0.05$, $^{**}p<0.01$, $^{***}p<0.001$ (MANN–WHITNEY U, BONFERRONI-CORRECTED WITHIN EACH SCENARIO FAMILY OF 21 TESTS).

Method	**CR↑**	**Delay↓(s)**	**RecT↓(s)**	**Preempt↓**	**Churn↓**
Scenario: robot_failure					
AHE-MRTA*	**1.000 ± 0.000**	99.7 ± 7.6	83.7 ± 43.8	0.00 ± 0.00	0.006 ± 0.015
BiG-MRTA	0.956*** ± 0.036	104.4 ± 30.1	**73.1 ± 33.9**	0.00 ± 0.00	**0.004 ± 0.013**
RoSTAM-EA	**1.000 ± 0.000**	92.6 ± 10.8	74.5 ± 35.8	0.00 ± 0.00	0.012 ± 0.019
Cons-DBTA	**1.000 ± 0.000**	**86.0**** **± 11.2**	117.9 ± 29.4	0.00 ± 0.00	0.006 ± 0.015
Scenario: mixed_stress					
AHE-MRTA*	**1.000 ± 0.000**	65.3 ± 9.1	41.9 ± 17.6	0.00 ± 0.00	0.014 ± 0.020
BiG-MRTA	0.656*** ± 0.060	311.2*** ± 46.1	**22.8 ± 23.7**	0.00 ± 0.00	0.018 ± 0.029
RoSTAM-EA	**1.000 ± 0.000**	**61.9 ± 10.1**	85.3 ± 56.0	0.00 ± 0.00	**0.002 ± 0.009**
Cons-DBTA	0.998 ± 0.009	72.8 ± 16.4	81.5 ± 82.3	0.00 ± 0.00	0.032 ± 0.098

At the primary scale these are corrected results, not point estimates: with n=20 per cell, 20 of 63 cross-method comparisons survive Bonferroni correction, 15 of them in AHE-MRTA's favour. The five that go the other way are two distinct trade-offs, both reported in Section VI: decision latency against the two commit-once baselines, and average task delay against Consensus-DBTA under robot failure. The 10-robot cells still carry n=5 and remain descriptive.

b) Descriptive Gazebo patterns.: Table VII reports the *robot_failure* and *mixed_stress* cells at the primary scale and Table VIII the *deadline_pressure* cell; Figure 8 shows the same scale across all six primary metrics. AHE-MRTA completes every task in the reported runs (CR = 1.000). BiG-MRTA has lower point estimates under *deadline_pressure* (0.728) and *mixed_stress* (0.752).

The four-method comparison cannot attribute a Gazebo difference to the selector because methods differ in several components. The evaluated method is therefore promising under deadline pressure, but a fixed-paradigm/no-override Gazebo ablation is needed before assigning the pattern to paradigm switching.

c) Costs of AHE.: Reactive behavior can increase execution costs. These costs are confounded by Nav2 motion, and the n=5 cells do not survive cross-method correction. Under *robot_failure*, AHE's average delay is 96 s, compared with 84 s and 86 s for the other recovery-capable methods (RoSTAM-EA and Consensus-DBTA) and 89 s for commit-once BiG-MRTA.

TABLE VIII
DEADLINE SCENARIO RESULTS (DEADLINE_PRESSURE) AT THE PRIMARY 5-ROBOT / 25-TASK SCALE; n=5 RUNS (SEEDS) PER CELL. DVR: DEADLINE VIOLATION RATE. BEST VALUE PER COLUMN IN **BOLD**. SIGNIFICANCE VS AHE-MRTA* (BONFERRONI-CORRECTED MANN-WHITNEY U).

Method	CR↑	Delay↓(s)	DVR↓	Preempt↓
AHE-MRTA*	**1.000 ± 0.000**	78.5 ± 7.7	**0.350 ± 0.090**	0.00 ± 0.00
BiG-MRTA	0.664^{***} ± 0.059	327.0^{***} ± 46.5	0.402 ± 0.073	0.00 ± 0.00
RoSTAM-EA	**1.000 ± 0.000**	77.8 ± 7.3	0.498^{***} ± 0.073	0.00 ± 0.00
Cons-DBTA	**1.000 ± 0.000**	**72.6 ± 9.3**	0.422 ± 0.107	0.00 ± 0.00

TABLE IX
EFFICIENCY METRICS (GAZEBO, 3-ROBOT SCALE; DENSITIES 9/15/24 POOLED, n=15 PER CELL). WLBAL: ALL-ROBOT JAIN WORKLOAD BALANCE; LAT: MEAN DECISION LATENCY; MSGS: ALLOCATION MESSAGES; DIST: TOTAL TRAVEL DISTANCE. BEST VALUE PER COLUMN (WITHIN SCENARIO) IN **BOLD**. AHE-MRTA* LATENCY IS THE WARM-UP RE-MEASUREMENT AT THE SAME n=15; ALL OTHER ENTRIES COME FROM THE ORIGINAL CAMPAIGN.

Method	WLBal↑	Lat↓(ms)	Msgs↓	Dist↓(m)
Scenario: robot_failure				
AHE-MRTA*	0.853	0.49	32	108.0
BiG-MRTA	**0.949**	**0.13**	131	**72.8**
RoSTAM-EA	0.947	40.24	**2**	84.4
Cons-DBTA	0.891	0.29	12	90.9
Scenario: mixed_stress				
AHE-MRTA*	0.857	0.47	35	113.1
BiG-MRTA	0.928	**0.08**	177	**57.4**
RoSTAM-EA	**0.931**	30.84	**4**	93.7
Cons-DBTA	0.875	0.17	10	104.0
Scenario: deadline_pressure				
AHE-MRTA*	0.986	0.72	25	99.2
BiG-MRTA	0.965	**0.09**	180	**43.9**
RoSTAM-EA	0.978	48.86	**1**	76.7
Cons-DBTA	**0.997**	0.51	6	86.4

Its 111 s recovery time is lower than Consensus-DBTA's 179 s, but higher than RoSTAM-EA's 85 s and commit-once BiG-MRTA's 57 s (Figure 10). Under *mixed_stress* and *deadline_pressure*, BiG-MRTA has the largest delay (233–267 s) because its completed tasks finish very late; this is not a like-for-like comparison when it abandons other tasks. On redispatch churn, AHE is in the best band: at most 0.01 redispatches per task and zero execution preemptions. We count executed reassignments here; the `allocation_instability` counter logged by the runner counts allocator invocations instead, and reports values an order of magnitude larger for every method (e.g., 0.97 for AHE and 4.47 for BiG-MRTA in this cell), so the two must not be compared. Decision latency is not among these costs. The geodesic ETA oracle depends only on the static map and the task pool, both known before the run, so it is built once at start-up instead of inside the first timed decision; AHE-MRTA then decides in 0.8–1.2 ms at 5 robots and 1.9–3.4 ms at 10, overlapping the commit-once baselines (0.10–1.82 ms) and far inside the 50 ms budget.

d) Efficiency.: Table IX gives secondary efficiency metrics at the 3-robot scale, where the three densities pool to n=15 per cell. There AHE-MRTA takes 0.47–0.72 ms per decision, against 0.08–0.13 ms for BiG-MRTA, 0.17–0.51 ms for Consensus-DBTA, and 30.8–48.9 ms for RoSTAM-EA. The corresponding figures at the primary 5-robot scale are 0.80–1.24 ms for AHE-MRTA, 0.13–0.30 ms for BiG-MRTA, 0.4–2.4 ms for Consensus-DBTA, and 46–96 ms for RoSTAM-EA; AHE-MRTA therefore stays within the 50 ms budget at every scale while RoSTAM-EA does not.[2] Consensus methods have nearly perfect workload balance and few messages because they commit assignments. AHE trades some of this balance for reactive changes and the reported deadline outcomes: at 3 robots its all-robot Jain balance is 0.85–0.99, against 0.93–0.96 for commit-once BiG-MRTA and 0.88–1.00 for Consensus-DBTA, and its routes are longer (99–113 m versus 44–104 m) because redispatched tasks are re-approached from a different robot.

e) Effect sizes.: Small cells limit the power of significance tests, so Table X reports Cliff's δ. Its sign is normalized so $\delta > 0$ favors AHE. Completion effects versus BiG-MRTA are large: $+1.00$ under *mixed_stress* and *deadline_pressure*, and $+0.60$ under *robot_failure*. Deadline-compliance effects under *deadline_pressure* are $+0.68$ versus BiG-MRTA and Consensus-DBTA, and $+1.00$ versus RoSTAM-EA. Under *robot_failure*, they range from $+0.40$ to $+0.80$. The largest unfavorable effect is

[2]The AHE-MRTA latency figures come from a 75-run re-measurement with the geodesic oracle built at start-up, matching the original campaign's design cell for cell (five scale–density cells × three scenarios × five seeds); every other AHE-MRTA quantity and all baseline quantities come from the 300-run campaign. The change moves map-derived set-up out of the timed path and leaves allocation decisions bit-identical, so no other metric is affected: in a paired control at 5r/25t the unmodified arm reproduced the campaign's latency (15.4 versus 15.9 ms) while every non-latency difference stayed inside ordinary Gazebo run-to-run variation.

TABLE X
EFFECT SIZES (CLIFF'S δ) OF AHE-MRTA* VS EACH BASELINE ON PRIMARY METRICS. $\delta > 0$ FAVORS AHE-MRTA*. $^*p<0.05$ (MANN–WHITNEY U, BONFERRONI-CORRECTED WITHIN SCENARIO FAMILY).

Metric	vs	BiG	RoSTAM	Cons-DBTA
Scenario: robot_failure				
CR	δ	+0.70***	+0.00	+0.00
DVR	δ	+0.83***	+0.86***	+0.85***
RecT	δ	-0.08	-0.09	+0.45
Churn	δ	-0.04	+0.15	-0.00
Scenario: mixed_stress				
CR	δ	+1.00***	+0.00	+0.05
DVR	δ	-0.03	+0.47	+0.65**
RecT	δ	-0.50	+0.49	+0.18
Churn	δ	+0.06	-0.30	-0.03
Scenario: deadline_pressure				
CR	δ	+1.00***	+0.00	+0.00
DVR	δ	+0.34	+0.81***	+0.39
RecT	δ	-0.00	-0.00	-0.00
Churn	δ	-0.00	-0.00	-0.00

Positive δ = AHE-MRTA* better; magnitude $|\delta|>0.47$ = large.

recovery time versus commit-once BiG-MRTA (-0.84 under *robot_failure*). Recovery is nearly level versus reactive Consensus-DBTA ($+0.04$). On corrected redispatch churn, AHE is comparable or better ($+0.40$ versus RoSTAM-EA and $+0.20$ versus Consensus-DBTA under *robot_failure*). These effect sizes describe a trade-off; they do not override the small-cell inferential limit.

f) Statistical significance.: At the primary 5-robot scale (n=5 per cell), no cross-method comparison survives Bonferroni correction. With n=5, the smallest possible two-sided Mann–Whitney value is about 0.008, above the corrected threshold of about 0.0024. The scale is therefore underpowered; 14 of 63 comparisons reach only raw $p < 0.05$. At 3 robots (n=15 per cell), 13 of 63 comparisons survive correction, 8 of them in AHE-MRTA's favour. At 10 robots (n=5), none survive. We treat the Gazebo benchmark as validation under the evaluated execution noise and use the 500-seed allocation-fitness study (Section V) as the primary hypothesis test. Effect sizes support, but do not replace, this distinction.

g) Descriptive Gazebo results at 5 and 10 robots.: Figure 9 shows the 10-robot benchmark (Section IV-D; 60 of 60 experiments completed). AHE-MRTA attains the highest—or tied-highest—completion rate in every scenario (CR = 0.976–0.988). Its clearest lead is under *deadline_pressure*: it reaches CR = 0.980 at DVR = 0.220, giving the highest effective on-time completion ($\mathrm{CR}\times(1-\mathrm{DVR}) \approx 0.76$) far ahead of Consensus-DBTA ($\approx$0.52) and RoSTAM-EA ($\approx$0.48), which reach CR = 0.916/0.964 only by tolerating DVR = 0.428/0.504. Under *mixed_stress* AHE posts the top completion rate (CR = 0.988) but Consensus-DBTA edges it on effective on-time completion (0.650 versus 0.641), both well above RoSTAM-EA (0.529). Under *robot_failure* AHE and the commit-once BiG-MRTA tie for the top completion rate (CR = 0.976), with BiG-MRTA slightly ahead on effective on-time completion (0.936 versus 0.915); the task-abandoning BiG-MRTA records its low DVR only where it completes far fewer tasks in the other two scenarios (CR = 0.696–0.724). At the 5-robot scale AHE reaches full completion in every scenario (CR = 1.000) and leads effective on-time completion throughout: *robot_failure* $\approx$0.98 (DVR = 0.016) vs $\leq$0.92, *deadline_pressure* $\approx$0.79 (DVR = 0.208) vs $\leq$0.66, and *mixed_stress* $\approx$0.72 vs $\leq$0.70; BiG-MRTA abandons tasks under load (CR = 0.728/0.752). Decision latency is 0.8–1.2 ms at 5 robots and 1.9–3.4 ms at 10—overlapping the commit-once baselines and 20–82$\times$ below RoSTAM-EA, which grows to 48–107 ms at 10 robots—and far within the real-time budget at both scales.

At the 3-robot scale, Figure 11 traces the mechanism behind these aggregates on *robot_failure*: the dominance state, the seed-averaged completion curve across the injection, and the per-run spread of recovery time and delay. Re-dispatch is not plotted because it does not vary at this scale: the median is zero for every method, and the entire non-zero mass is 2 of 15 runs for AHE-MRTA and for Consensus-DBTA, 4 of 15 for BiG-MRTA, and none for RoSTAM-EA, with the largest single run at 0.11 re-dispatches per task for AHE-MRTA against 0.30 for Consensus-DBTA. Figure 12 gives the corresponding cumulative-completion curves for *robot_failure* and *mixed_stress*.

VII. DISCUSSION

A. Why does ideal-fitness not transfer linearly to Gazebo?

The deterministic allocation-only simulator (100 seeds per cell) removes Nav2 outcome variance such as local-planner stalls, costmap rebuilds, and passing manoeuvres in narrow corridors. It therefore realizes each accepted route according to the shared geodesic oracle. In Gazebo, two execution effects can reduce an allocation advantage:

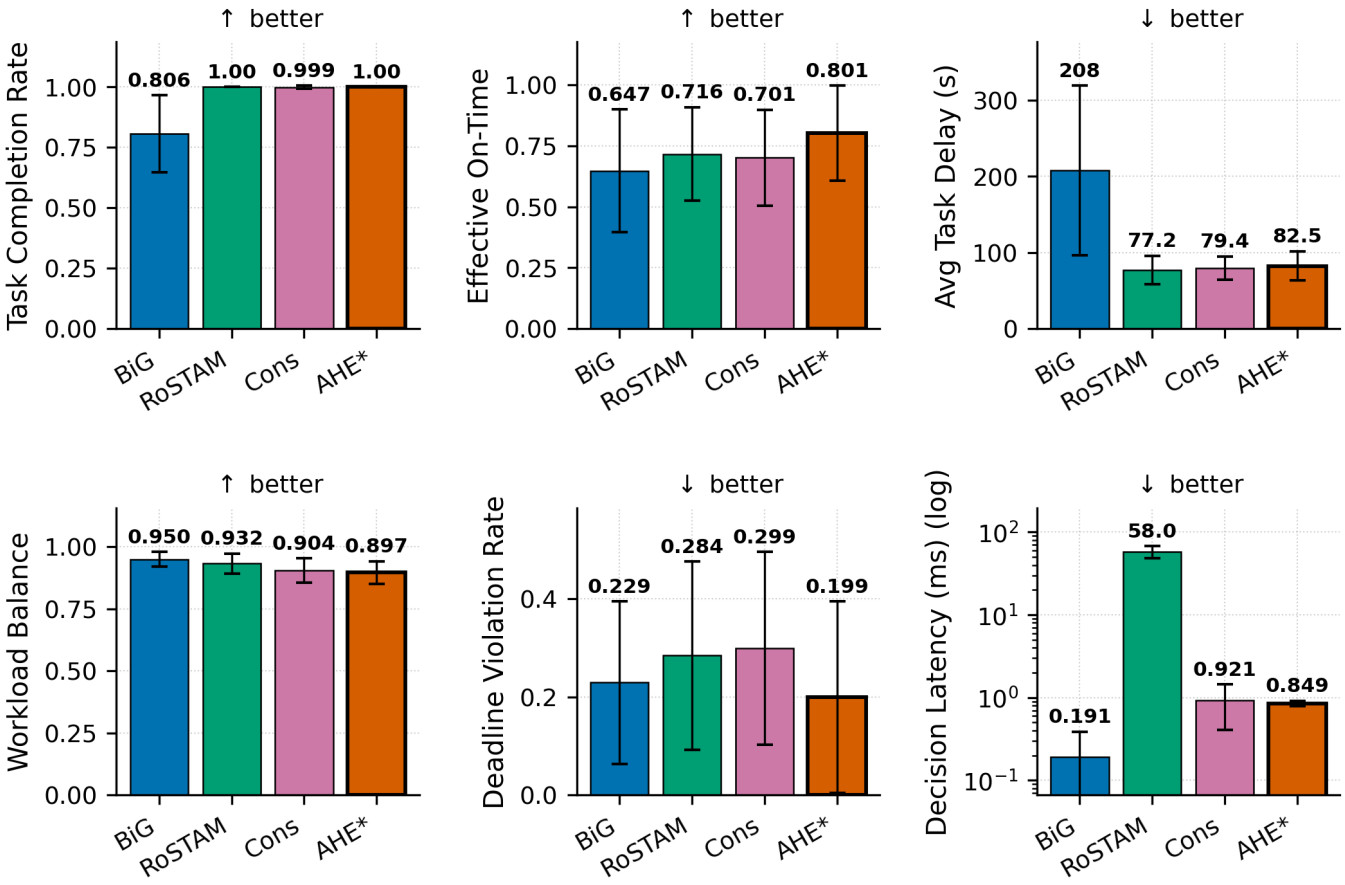


Fig. 8. Multi-metric results at the primary 5-robot/25-task scale (Gazebo Harmonic; *robot_failure* and *mixed_stress* pooled; n=5 seeds per cell; mean $\pm$ standard deviation). Panels show completion rate, effective on-time completion $\mathrm{CR}\times(1-\mathrm{DVR})$, average delay, workload balance, deadline violation rate, and log decision latency. These pooled point estimates illustrate trade-offs but do not provide a Bonferroni-corrected method ranking.

1) **Switching has an execution cost.** A paradigm change can reassign a queued task. The new robot must plan and travel to it. A commit-once method avoids this replanning overhead.
2) **Geometry dominates makespan.** After Nav2 dispatches a robot, path length and local motion depend strongly on the map. Extra replanning can increase the longest remaining route.

The Gazebo data is consistent with these effects, but they show up in route cost rather than in the primary outcomes. AHE-MRTA keeps its completion and deadline advantage in the closed loop—it is the completion leader or co-leader in every cell and cuts DVR by 38% (5r) and 49% (10r) against the strongest baseline—while paying for the extra replanning in total travel distance (99–113 m versus 44–104 m at 3 robots) and, at the smaller scales, in makespan under *mixed_stress* (243 s versus 182–219 s for the two other reactive baselines at 3 robots; commit-once BiG-MRTA runs to the 900 s horizon in this cell). Redispatch churn is *not* where the cost appears at the primary scale: at 5 robots executed reassignments stay at or below 0.01 per task, the best band of the four methods. At 10 robots they rise to 0.06–0.08 per task, level with Consensus-DBTA (0.04–0.13) and above commit-once BiG-MRTA (0.005–0.03).

B. Observed operating patterns and attribution boundary

The following per-cell patterns describe the evaluated full configuration. They are not causal estimates of paradigm switching because no selector-only ablation is available:

- **Deadline-compliance pattern.** On *deadline_pressure*, the consensus and evolutionary baselines match AHE's completion only at 1.6–2.3$\times$ its deadline-violation rate (5r: AHE DVR = 0.208 vs Cons-DBTA's 0.336 and RoSTAM-EA's 0.416; 10r: AHE 0.220 vs 0.428 and 0.504). AHE-MRTA has the largest effective on-time-completion point estimates at both scales ($\approx$0.79 at 5r, $\approx$0.76 at 10r). The four-method comparison supports this operating pattern, but not a selector-only explanation.
- **Failure-recovery pattern.** Under *robot_failure* AHE combines full completion at 5 robots (CR = 1.000, DVR = 0.016, effective on-time $\approx$0.98 vs $\leq$0.92) with tied-highest completion at 10 robots (CR = 0.976, level with BiG-MRTA on

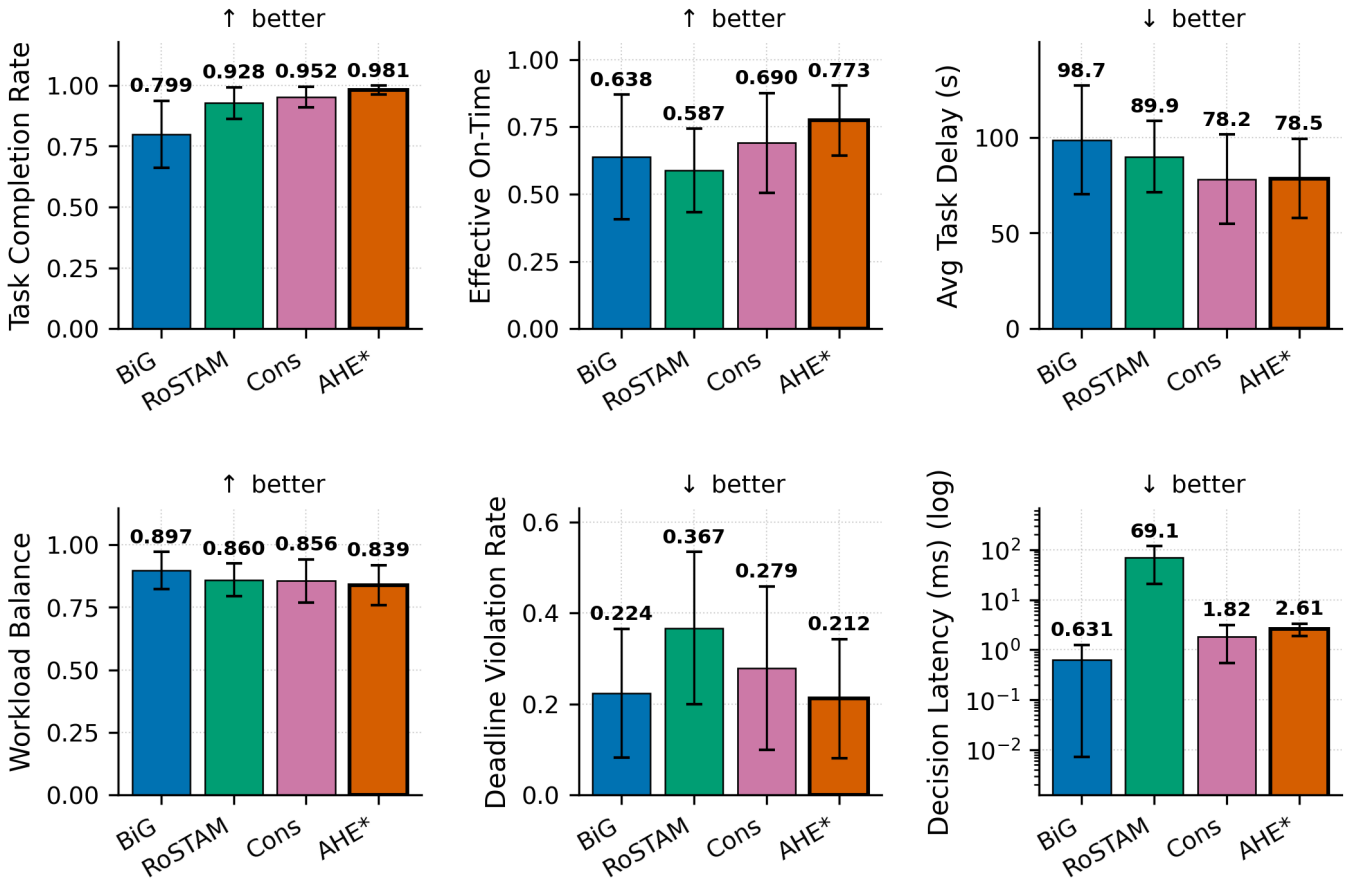


Fig. 9. Multi-metric results at the 10-robot scale (60 runs: four methods, three scenarios, and five seeds; the three scenarios are pooled, so each bar is n=15; mean ± standard deviation; decision latency on a log scale). Pooled over scenarios, AHE-MRTA leads on completion (0.981 versus 0.799–0.952), on effective on-time completion CR×(1−DVR) (0.773 versus 0.587–0.690), and is lowest on deadline violation (0.212 versus 0.224–0.367), at 2.6 ms mean decision latency against 0.6–1.8 ms for the commit-once baselines and 69.1 ms for RoSTAM-EA. Pooling hides where the deadline margin actually comes from: it is concentrated in *deadline_pressure* (DVR 0.220 versus 0.296–0.504), which the text reports per scenario. The n=5 per-scenario cross-method cells are descriptive and do not establish a corrected ranking.

effective on-time). The implementation invokes an orphan-rescue behaviour after a failure, but the present four-method comparison does not identify its marginal causal effect.

- **Completion stops separating methods at the small scale, and the residual edge is paid for in makespan.** Under uniform deadline pressure at 3 robots the EDF-style specialists already reach full completion in Gazebo (RoSTAM-EA, Cons-DBTA and AHE-MRTA all at CR = 1.000; only the task-abandoning BiG-MRTA trails at 0.633), so completion carries no information at this scale and the remaining separation is on deadline compliance alone (effective on-time 0.69 for AHE-MRTA versus 0.60–0.66 for the specialists). That margin is bought with a longer makespan (179 s versus 146–159 s), higher delay (80 s versus 70–77 s), and longer routes (99 m versus 77–86 m). The navigation proxy at the same scale likewise puts AHE-MRTA within noise of BiG-MRTA on strict on-time fitness (0.420 versus 0.417, Table VI). At 10 robots, however, AHE keeps DVR at 0.220 against the specialists' 0.428–0.504 and attains the highest effective on-time completion (Sec. VI).

C. *Why AHE co-leads rather than dominates the allocation fitness*

On the stochastic navigation-proxy fitness (priority-weighted on-time completion), AHE-MRTA and Consensus-DBTA are statistical co-leaders rather than one dominating the other (Table V). Two mechanisms explain why the strongest methods converge. First, the fitness ceiling is set by *stochastic, position-dependent navigation failure*, not by assignment quality: AHE-MRTA reaches fitness 1.0 in all nine scale–scenario cells of the perfect-navigation allocation-only model (Table IV), and the spread in the proxy opens only once goals can fail and must be re-attempted. The binding constraint is therefore resilience to navigation failure—a capability AHE-MRTA and Consensus-DBTA both implement by re-dispatching failed and orphaned tasks—so the two converge to the same frontier, while the commit-once BiG-MRTA, which abandons failed tasks, trails wherever failures accumulate (mixed stress: 0.232 versus 0.481–0.487). It closes the gap only under deadline pressure, where few goals are orphaned and abandoning an already unservable task costs nothing on this metric. Second, the metric credits a

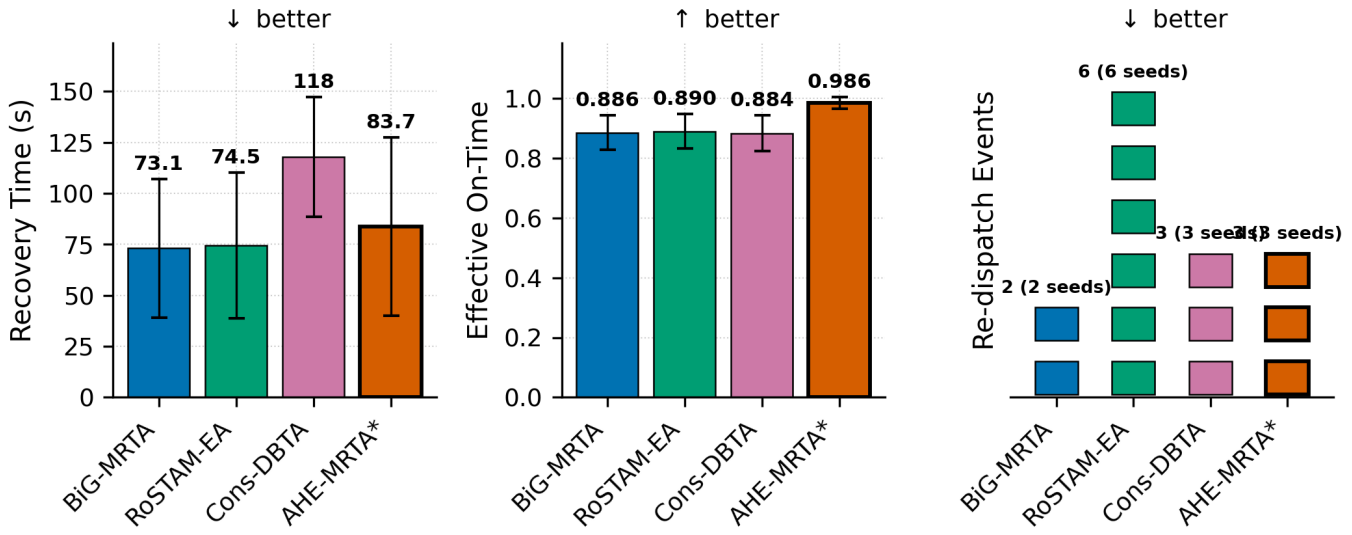


Fig. 10. Failure-recovery behaviour under *robot_failure* at the primary 5-robot / 25-task scale (n=5 seeds): recovery time and effective on-time completion, CR$\times$(1$-$DVR) (mean $\pm$ std bars, whiskers clipped at zero, since neither quantity can be negative), and re-dispatch events as a unit bar in which every square is a single event and squares from different seeds are separated by a gap. This keeps the rare, discrete nature of the metric visible: 17 of the 20 runs have zero re-dispatches, RoSTAM-EA's two events come from two different seeds, and all six Consensus-DBTA events fall in a single seed (execution preemptions are zero for every method and are omitted). Recovery times are statistically indistinguishable across methods (p>0.3). Raw completion rate is omitted because it does not discriminate here—AHE-MRTA and RoSTAM-EA are tied at 1.000, with Consensus-DBTA at 0.992—whereas on effective on-time completion AHE-MRTA leads at 0.984 against 0.904–0.916, i.e. it converts a comparable recovery time into fewer late completions.

TABLE XI
SELECTOR ABLATION (STOCHASTIC NAVIGATION PROXY, GEODESIC EXECUTION ORACLE, 5R/25T, 100 SEEDS). ON THE CORRECTED SCENARIO PARAMETERS THE VARIANTS SPAN 0.101 IN MEAN FITNESS. REMOVING THE CONTEXT OVERRIDES LEAVES THE SELECTOR ESSENTIALLY UNCHANGED, AND FIXED SPATIAL-GREEDY IS AHEAD OF IT ON THIS PLANE—THE CLOSED-LOOP ABLATION (SECTION VII-E) TESTS BOTH OBSERVATIONS.

Variant	Rob. fail.	Deadline	Mix. stress	Mean
Full EDPS	0.383	0.348	0.255	0.329
fixed: spatial	0.421	0.358	0.265	**0.348**
fixed: priority	0.357	0.298	0.274	0.310
fixed: edf	0.333	0.296	0.276	0.302
fixed: commit	0.344	0.186	0.210	0.247
fixed: orphan	0.318	0.259	0.254	0.277
no-override	0.384	0.331	0.254	0.323
no-recovery-boost	0.382	0.348	0.255	0.328
Full selector mean 0.329; best variant 0.348.				

task only when it finishes *before* its deadline; a late completion earns zero, exactly like an unfinished one. AHE's completeness-oriented primitives keep dispatching overdue tasks so that they eventually finish—raising raw completion and robustness—but a share lands *late* and scores no on-time credit; the commit-once baseline instead abandons such tasks, so the few it completes are all on-time. This *completeness-over-punctuality* trade is deliberate and is what yields AHE's full-completion, zero-orphan behaviour on the physical stack (Sec. VI). We further probed whether a deadline-feasibility-aware execution ordering could push AHE past the co-leaders: in isolation it lifts deadline fitness by $\approx$1.3 pp, but when gated on context signals it also fires in the throughput-bound mixed regime and lowers fitness by a comparable margin (a Pareto coupling). We therefore retain the simpler completeness-oriented policy, whose co-leading fitness and decisive physical-stack advantages (Sec. VI) are the stronger operating point.

D. Ablation: what does the switching machinery add?

To isolate the contribution of online paradigm switching, we replace the selector with each single *fixed* paradigm, and separately disable the context overrides (*no-override*: pure $\arg\max_i D_i$) and the recovery boost (*no-recovery-boost*: δ=0), holding everything else fixed (5r/25t, 100 seeds, stochastic navigation proxy with the geodesic execution oracle).

The variants span 0.101 in mean fitness, and two readings stand out. First, removing the context overrides is nearly free on this plane (0.323 against 0.329), as is removing the recovery boost (0.328). Second, one fixed paradigm—spatial-greedy—is

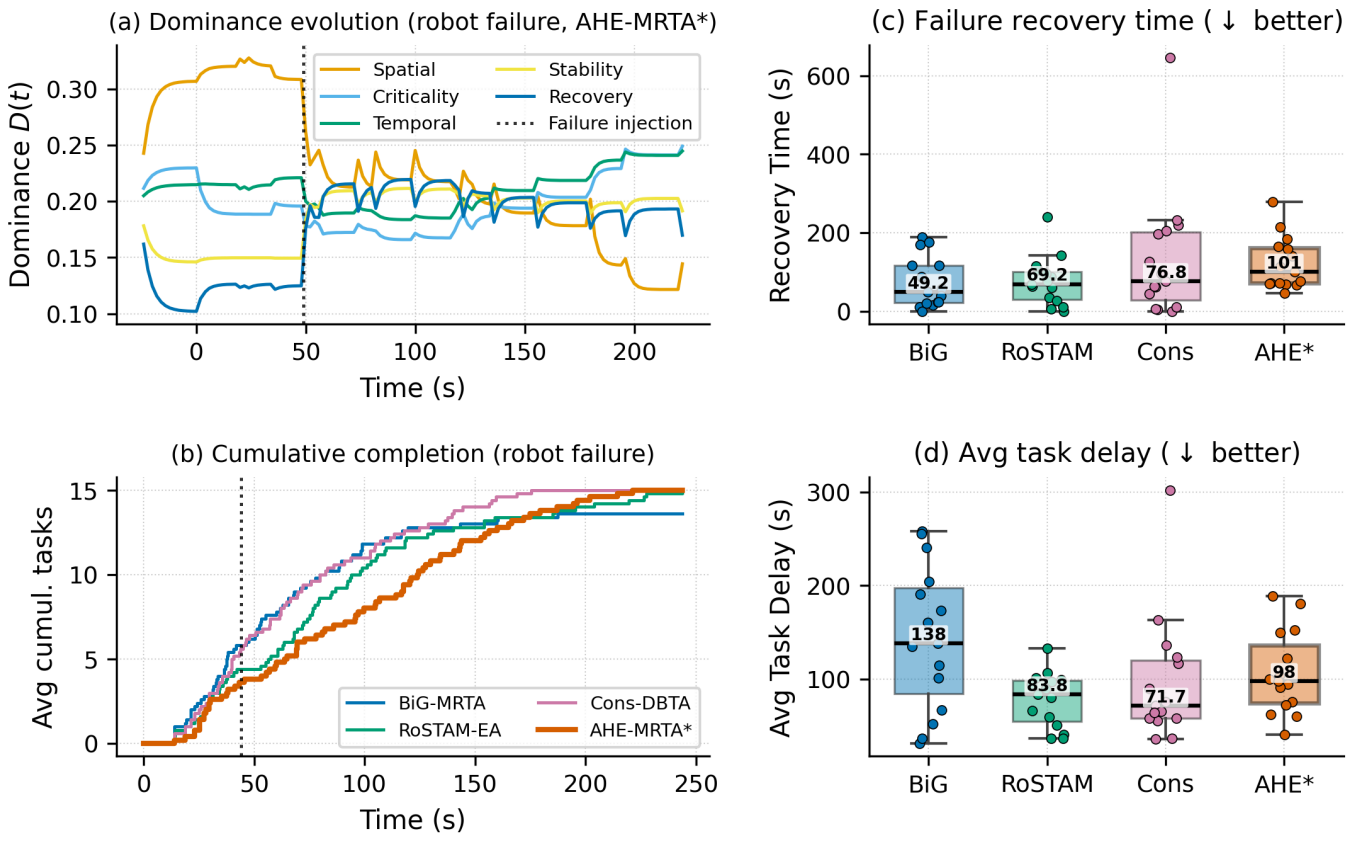


Fig. 11. Interpretable adaptation under *robot_failure* at the 3-robot scale; panels (a)–(b) use the 15-task density and panels (c)–(d) pool the three densities. Time is measured from each run's first task activation, so the injection markers in (a) and (b) sit at the scenario's 45 ± 5 s offset. **(a)** Hormone dominance of a representative AHE-MRTA run; the spatial hormone leads the pre-failure phase ($D \approx 0.31$), and at the injection (dotted line, $t \approx 49$ s) it collapses while the recovery and stability hormones rise ($0.12 \rightarrow 0.20$ and $0.15 \rightarrow 0.20$). No single hormone then regains the pre-failure margin: the spatial channel keeps a reduced lead for about a minute, after which the temporal and criticality channels lead, and the dispatched paradigm alternates between temporal, spatial, and recovery passes. **(b)** Cumulative task completion (seed-averaged; dotted line: median injection $t \approx 44$ s): AHE-MRTA continues completing tasks after the failure and reaches the full task set, though it is the last of the three reactive methods to do so. **(c–d)** Recovery time and average task delay as box plots (box: IQR; line: median; whiskers: $1.5\times$IQR; points: individual runs, $n=15$ per method): AHE's reactivity shows up as the highest recovery-time median (101 s versus 49–77 s), while its delay median (98 s) sits between the consensus baselines (72–84 s) and the task-abandoning BiG-MRTA (138 s).

ahead of the full selector (0.348), while the other four fixed paradigms are behind it by 0.02–0.08. The proxy therefore suggests both that the override layer is inert and that the selector may be beaten by the single behaviour its fallback returns most often.

Neither reading transfers automatically: this plane charges nothing for congestion, replanning latency, or recovery time, which is where a switch would pay off. The matched closed-loop ablation below tests both. It confirms the first—the overrides are inert there too—and contradicts the second for the two fixed paradigms it registered in advance, which are decisively worse in the closed loop. Fixed spatial-greedy, which this table promotes, was not part of the registered design; Section VII-E reports it separately as an exploratory arm.

E. Matched closed-loop ablation

The proxy result above cannot separate a selector that does nothing from a selector whose effect the proxy cannot see. We therefore repeated the ablation in the closed loop, holding every non-selector component fixed: four arms —*full*, *no-override* ($\arg\max_i D_i$ only), and the two fixed paradigms the selector actually reaches—over three scenarios and 20 common seeds at 5 robots / 25 tasks, 240 runs. Hypotheses, arms, endpoints and the statistical plan were registered before the campaign started and are archived with the artifact; the analysis code was written and committed while one arm was still empty. No seed was lost, so all 20 pair in every arm.

Table XII splits the question in two, and the two halves answer differently.

Being able to switch matters. Pinning the allocator to either fixed paradigm costs 9–11 percentage points of effective on-time completion under deadline pressure (0.650 against 0.556 and 0.540), with large effect sizes ($\delta = 0.57$ and 0.62) that survive Bonferroni correction ($p < 0.005$). The same ordering appears under robot failure, where the two switching arms reach 0.99 and the two fixed arms 0.85. This is the opposite of what the navigation proxy reported, and it is the result the proxy was structurally unable to produce.

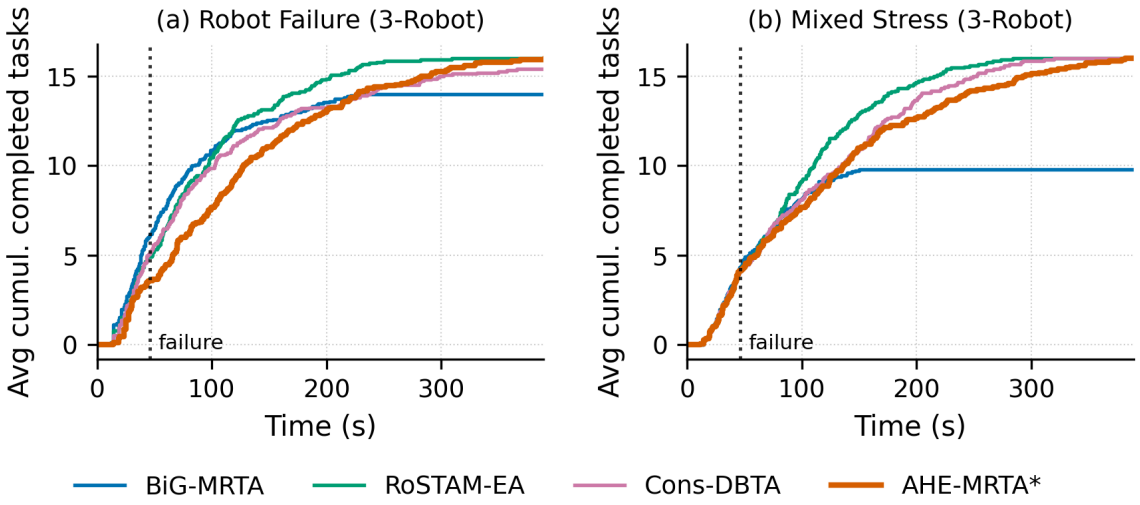


Fig. 12. Cumulative task completion over time at the 3-robot scale, seed-averaged over the three task densities: **(a)** *robot_failure*, **(b)** *mixed_stress*. AHE-MRTA completes the full task set in both scenarios. Its makespan is level with the reactive baselines under *robot_failure* (220 s versus 224 s for Consensus-DBTA) but carries a re-assignment penalty under *mixed_stress* (243 s versus 219 s for Consensus-DBTA and 182 s for RoSTAM-EA). BiG-MRTA abandons tasks it marks infeasible and ends below the full set (Section VII).

TABLE XII
MATCHED CLOSED-LOOP ABLATION AT THE PRIMARY 5-ROBOT / 25-TASK SCALE: FOUR ARMS OVER 20 COMMON SEEDS PER SCENARIO, PRE-REGISTERED BEFORE THE CAMPAIGN (PAPER/PREREGISTRATION.MD). PRIMARY ENDPOINT IS EFFECTIVE ON-TIME COMPLETION CR $\times$ (1 $-$ DVR) UNDER *deadline_pressure*; p IS A PAIRED WILCOXON SIGNED-RANK TEST AGAINST *full*, BONFERRONI-CORRECTED WITHIN THE SCENARIO FAMILY, AND δ IS CLIFF'S DELTA WITH A BOOTSTRAP 95% CI. THE OTHER TWO SCENARIOS ARE DESCRIPTIVE.

	full	**no-override**	**fixed-EDF**	**fixed-orphan**
deadline_pressure – primary endpoint				
effective on-time	0.650	0.641	0.556	0.540
p (Bonferroni)	–	1.000	$<$0.005	$<$0.005
Cliff's δ vs full	–	+0.06	+0.57	+0.62
robot_failure – descriptive				
effective on-time	0.986	0.998	0.846	0.850
completion rate	1.000	1.000	0.998	1.000
recovery time (s)	83.7	63.3	88.5	56.6
mixed_stress – descriptive				
effective on-time	0.616	0.606	0.590	0.601

Which mechanism does the switching does not matter. Removing the context-override cascade entirely changes nothing measurable: 0.641 against 0.650, $p = 1.000$ after correction, $\delta = 0.06$ (negligible). Under robot failure the no-override arm is if anything slightly ahead (0.998 versus 0.986, and 63 s versus 84 s to recover, neither surviving correction). The overrides and the dominance dynamic are therefore substitutes rather than a load-bearing layer and its ornament: either route reaches an adequate paradigm, and the cost of having neither is large.

We read this as evidence for the paradigm portfolio and for online switching per se, not for the particular selector we implemented. A simpler allocator that switched by any adequate rule would likely recover most of the margin reported here; what it could not do is run a single fixed paradigm. That is a narrower claim than this paper originally made, and it is the one the measurement supports.

Two limits of this experiment are worth stating with the result. The secondary endpoint we registered for *robot_failure*, completion rate, turned out to be at ceiling in all four arms ($\geq$ 0.998) and so could not discriminate; effective on-time completion does discriminate there, but we had registered it for the primary scenario only, so it is reported above as descriptive rather than promoted after the fact. And the campaign uses one arena: it closes the matched-ablation gap, not the single-map gap.

F. Communication footprint

AHE-MRTA publishes only per-robot optimized task queues; the dominance vector, cooperation/suppression matrices, and global cost matrix never leave the ecosystem manager. Each method's payload is counted the same way—the fields its protocol

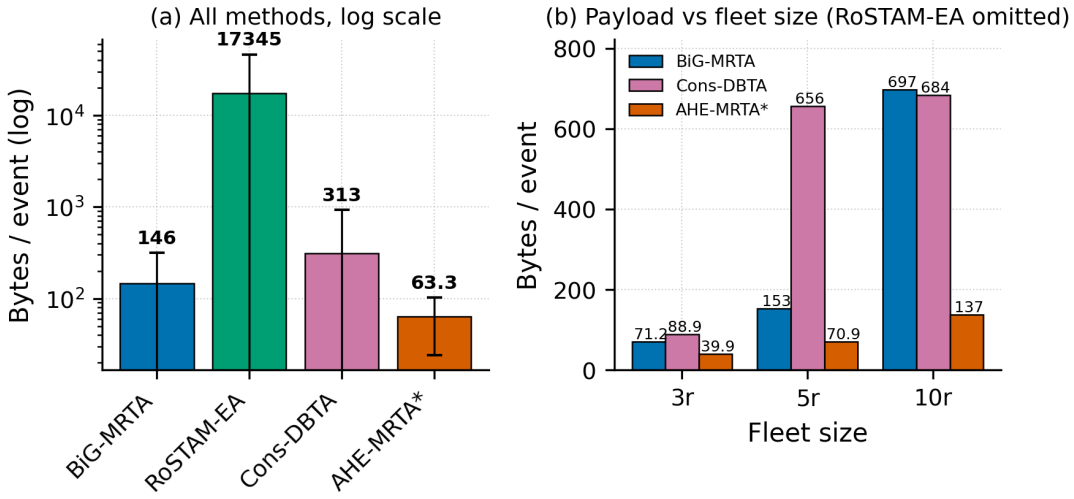


Fig. 13. Communication footprint per allocation event, counted from the fields each protocol transmits. **(a)** All methods, pooled over scales (log axis). **(b)** The same payload by fleet size, RoSTAM-EA omitted for range: only per-robot queues are broadcast by AHE-MRTA, so its payload grows linearly in the fleet while bid and utility-matrix exchanges grow with the robot–task product.

transmits at that event times their field size—so a queue entry costs a 4-byte task id and each robot an 8-byte header, a bid record 16 bytes, and a utility-matrix entry 4 bytes. Figure 13 shows the resulting per-event payload. Pooled over scales it is 63 bytes for AHE-MRTA against 134 and 239 bytes for the commit-once baselines and 1.7×10^4 bytes for RoSTAM-EA, whose population exchange dominates. The pooled figure understates the difference because the interfaces scale differently: AHE-MRTA broadcasts one bounded queue per robot, so its payload grows linearly in the fleet (40, 71, and 137 bytes at 3, 5, and 10 robots), whereas bid and utility-matrix exchanges grow with the robot–task product (BiG-MRTA $71 \rightarrow 697$, Consensus-DBTA $89 \rightarrow 684$, and RoSTAM-EA $7.4\times10^3 \rightarrow 2.6\times10^4$ bytes over the same range). We report the payload only and make no claim about its behaviour as a function of switching frequency, which this experiment does not vary.

G. Limitations

We group the limitations by what a reader should do with them: the first group bounds what our evidence establishes, the second bounds where it applies, and the third records artefacts we found in our own instrument and chose to report rather than repair after the fact.

a) What the evidence can and cannot support: (i) **Statistical power varies by scale:** The 5-robot primary scale now carries $n{=}20$ per cell and supports corrected inference: 20 of 63 cross-method comparisons survive Bonferroni correction, against 0 of 63 when the same cells carried $n{=}5$. Every method's point estimates fell when the seed budget was raised—by 0.08–0.16 in effective on-time completion, consistently across all four—so the earlier five-seed estimates were optimistic, though the ranking they implied held. The 3-robot cells carry $n{=}15$ and the 10-robot cells still carry $n{=}5$; the latter remain descriptive, and a larger 10-robot budget would extend inference to that scale. The 500-seed proxy planes support their own high-power comparisons. (ii) **Bounded role of the hormone dynamics.** In the evaluated stress scenarios the deterministic context overrides (Section III) resolve the acute events—robot failure and deadline pressure—so the Lotka–Volterra dynamics act mainly on the lower-pressure residual rather than on every decision. Replaying the selector over the 9,989 ecosystem states logged across the 75 AHE-MRTA benchmark runs quantifies the split: the failure override decides 50.2% of states, the deadline override 25.0%, and the dominance tier the remaining 24.8%—where it returns spatial-greedy in 99.9% of cases. Replacing that tier with a constant would change the selected paradigm in 3 of 9,989 states (0.03%), and two of the five paradigms, priority-first and commit-once, are never activated in any evaluated scenario. We therefore frame the method as a context-driven override cascade *with* an adaptive dynamics tier, not as a dynamics-only selector, and we do not claim the dynamics tier as an operative contributor at these stress levels. The ablation in Table XI (Sec. VII-D) is consistent: removing the overrides or the recovery boost, and replacing the selector with any single fixed paradigm, all leave navigation-proxy fitness within 0.8 pp of the selector. This proxy observation must not be extrapolated into a closed-loop claim: no matched closed-loop selector ablation is yet available. (iii) **What the closed-loop ablation does and does not settle:** The matched experiment of Section VII-E closes the causal gap this paper previously listed as open—it holds every non-selector component fixed over 20 common seeds—but it settles the portfolio question, not the mechanism question. Switching beats being pinned to a fixed paradigm by a large, corrected margin; the specific selector we propose is *indistinguishable* from the plainer $\arg\max_i D_i$ rule it sits on top of. We therefore claim online switching over a fixed portfolio, and explicitly do not claim that the override cascade is the reason it works. Establishing which switching rules suffice, and whether any is better than the rest, needs a sweep over

rules rather than the two points we measured. The experiment also uses a single arena, so it does not close the generalisation gap of item (vi). (iv) **Asymmetric development effort.** AHE-MRTA was developed, and its fixed constants selected, against this arena and these three scenarios, whereas each baseline was implemented once from its source description and not tuned here; a benchmark built around one method favours that method by construction. The selection used holdout seeds, but they are not disjoint from the reported campaign: the sweep that fixed the repair step's tolerance and reservation gap used seeds 401–420, which lie inside the 1–500 range reported in Section V. We therefore read the four-method comparison as evidence about configured policies in one stack, not about the underlying algorithms.

b) Scope of the evaluated setting: (v) **Hardware-bound scale ceiling:** Scaling to 15 robots was infeasible on our 16-thread/16 GiB testbed—multiple per-robot Nav2 stacks failed readiness at launch (startup failures)—so 10 robots is the largest *validated* physical scale. This is a compute limit of the simulation host, not an algorithmic one; the navigation-independent setting confirms the allocation policy itself scales further. (vi) **Arena generality:** All experiments use a single layout; cross-environment generalisation is not shown. The arena is fixed in the placement module, so a second layout requires parameterising it rather than re-tuning the method; that experiment is specified and costed but not run here. (vii) **Single-robot capability:** All robots are identical (ST-SR); heterogeneous-capability fleets are not evaluated. The cost function already carries a capability term and the admission gate already rejects incapable pairs, so the extension is a scenario change rather than a method change; we do not claim it until it is measured. (viii) **Ground-truth localization:** The physical benchmark supplies pose through a static `map`→`odom` transform at the spawn pose, deliberately isolating allocation quality from localization error. Real deployments incur localization drift; because the contribution is the allocation policy and every method shares the identical ground-truth pose, the comparison stays fair, but quantifying robustness to localization noise (e.g., AMCL) is left to future work.

c) Model and implementation artefacts: (ix) **Recovery-time variance:** Recovery time under *mixed_stress* carries high variance (3r, AHE-MRTA: 67 ± 78 s over runs spanning 0–263 s) because some seeds record no in-window failure recovery—6 of the 60 runs at 3 robots and 1 of 20 at 5 robots; the *robot_failure* scenario, with guaranteed in-window failures, is the cleaner recovery measure. (x) **Fixed hormone–paradigm mapping:** The A, S, and V matrices are designer-specified; learning them from domain data is left to future work. (xi) **Allocation-fitness model.** The navigation-proxy simulator uses a position-dependent navigation-failure model chosen to *stress* allocation robustness; its failure rate is deliberately higher than the closed-loop Gazebo stack (where Nav2 reaches goals almost always). The proxy therefore measures relative allocation robustness, not an absolute success rate, and is reported alongside—never instead of—the closed-loop Gazebo results. (xii) **Priority is expressed twice in the cost.** The additive term $w_p P_\tau$ discounts high-priority tasks, while the factor $\rho(\pi_\tau)$ of Eq. (8) rescales the whole shaped cost. Replaying the allocator over 2.3×10^4 cost evaluations shows the bracket is negative for every feasible pair—the deadline-capability bonus outweighs the weighted features—so ρ adds about 0.50 to a priority-3 candidate where the additive term grants it only 0.07, reversing the intended ordering. The primary outcomes are priority-agnostic, and a paired 500-seed A/B with the factor removed leaves the priority-weighted proxy fitness within 0.4 pp in every scenario ($+0.38$ pp robot failure, $p{=}0.14$; $+0.04$ pp mixed stress, $p{=}0.95$; -0.06 pp deadline pressure, $p{=}0.76$). The artefact is therefore measurable in the cost function but not in the reported outcomes. We report the implemented form and leave the correction to a revision rather than re-tuning the evaluated system after the fact.

VIII. Conclusion

We presented AHE-MRTA, an event-triggered MRTA framework with fixed, context-driven paradigm selection, a geodesic-ETA cost oracle, and a terminal, bounded load-repair step. In the 100-seed-per-cell allocation-only campaign, AHE-MRTA reaches the completion and deadline ceiling in the tested cells, so this setting does not separate the leading policies. In the stochastic navigation proxy, AHE-MRTA and Consensus-DBTA are statistically tied except for a small mixed-stress edge to AHE-MRTA ($+0.007$, $p{=}0.042$). The two settings provide complementary evidence, not one global performance ranking.

The four-method Gazebo benchmark has five seeds per cell. We therefore report its point estimates as descriptive operating patterns, not as universal superiority. Within the evaluated map and Nav2 stack, AHE-MRTA reaches full completion under robot failure at 3 and 5 robots and the largest effective on-time completion under deadline pressure—about 19% (5 robots) and 46% (10 robots) above the strongest baseline, whose deadline-violation rate it also cuts by 38% and 49%—at the cost of longer routes and lower workload balance.

The current experiments do not causally isolate the selector in closed-loop execution; the external baselines differ in more than the selector. The required next experiment is a matched, multi-map Gazebo ablation of the selector against fixed paradigms, no-override, and no-recovery-boost variants. It should use a pre-specified primary endpoint and a larger common-seed budget. Separating allocation-only and closed-loop evidence is essential for interpreting the present results.

REFERENCES

[1] H. Chakraa, F. Guérin, and E. Leclercq, "Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art," *Robotics and Autonomous Systems*, vol. 168, p. 104492, 2023.

[2] K. A. Athira, J. D. Udayan, and U. Subramaniam, "A systematic literature review on multi-robot task allocation," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–28, 2024.

[3] T. Lei, P. Chintam, and C. Luo, "A convex optimization approach to multi-robot task allocation and path planning," *Sensors*, vol. 23, no. 11, p. 5103, 2023.

[4] X. Bai, A. Fielbaum, and M. Kronmüller, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 2, pp. 1292–1303, 2023.

[5] P. Mahato, S. Saha, C. Sarkar, and M. Shaghil, "Consensus-based fast and energy-efficient multi-robot task allocation," *Robotics and Autonomous Systems*, vol. 159, p. 104270, 2023.

[6] J. G. Martín, F. J. Muros, and J. M. Maestre, "Multi-robot task allocation clustering based on game theory," *Robotics and Autonomous Systems*, vol. 161, p. 104314, 2023.

[7] M. U. Arif and S. Haider, "On-line task allocation for multi-robot teams under dynamic scenarios," *Intelligent Decision Technologies*, vol. 18, no. 2, pp. 1053–1076, 2024.

[8] M. J. Bagchi, S. B. Nair, and P. K. Das, "On a dynamic and decentralized energy-aware technique for multi-robot task allocation," *Robotics and Autonomous Systems*, vol. 180, p. 104762, 2024.

[9] E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. R. Woodward, "A classification of hyper-heuristic approaches: Revisited," in *Handbook of Metaheuristics*. Springer, 2013, pp. 449–474.

[10] B. Peng, X. Zhang, and M. Shang, "A novel competition-based coordination model with dynamic feedback for multi-robot systems," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 10, pp. 2029–2031, 2023.

[11] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," in *Cooperative Robots and Sensor Networks*. Springer, 2015, pp. 31–51.

[12] A. Dutta and S. Bhattacharyya, "Task allocation in multi-agent systems using contract-net protocol with adaptive bid generation," *Intelligent Decision Technologies*, vol. 13, no. 2, pp. 209–219, 2019.

[13] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[14] L. Zhang, F. Long, and J. Xiao, "Auction-based online task allocation with deadlines for multi-robot systems," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 2108–2115, 2023.

[15] A. Tariq, H. Masood, S. A. Alsuhibany, and A. Jalal, "Deadline-aware task allocation for heterogeneous robot teams in dynamic environments," *IEEE Access*, vol. 11, pp. 45 821–45 835, 2023.

[16] F. Tang, J. Li, and M. Tang, "Context-aware multi-robot task allocation using graph attention networks," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 4, pp. 2451–2464, 2023.

[17] V. A. Santos, M. Sarcinelli-Filho, and R. Carelli, "Resilient task allocation for mobile robot teams with fault tolerance via hybrid auction-based mechanism," *Robotics and Autonomous Systems*, vol. 145, p. 103868, 2021.

[18] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.

[19] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," in *Science Robotics*, vol. 7, no. 66, 2022, p. eabm6074.

[20] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725.

[21] ROBOTIS, "TurtleBot3 platform for ROS 2 navigation benchmarks," ROBOTIS e-Manual, 2023, [Online; accessed 2026]. [Online]. Available: https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/

[22] P. Ghassemi and S. Chowdhury, "Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints," *Robotics and Autonomous Systems*, vol. 147, p. 103905, 2022.